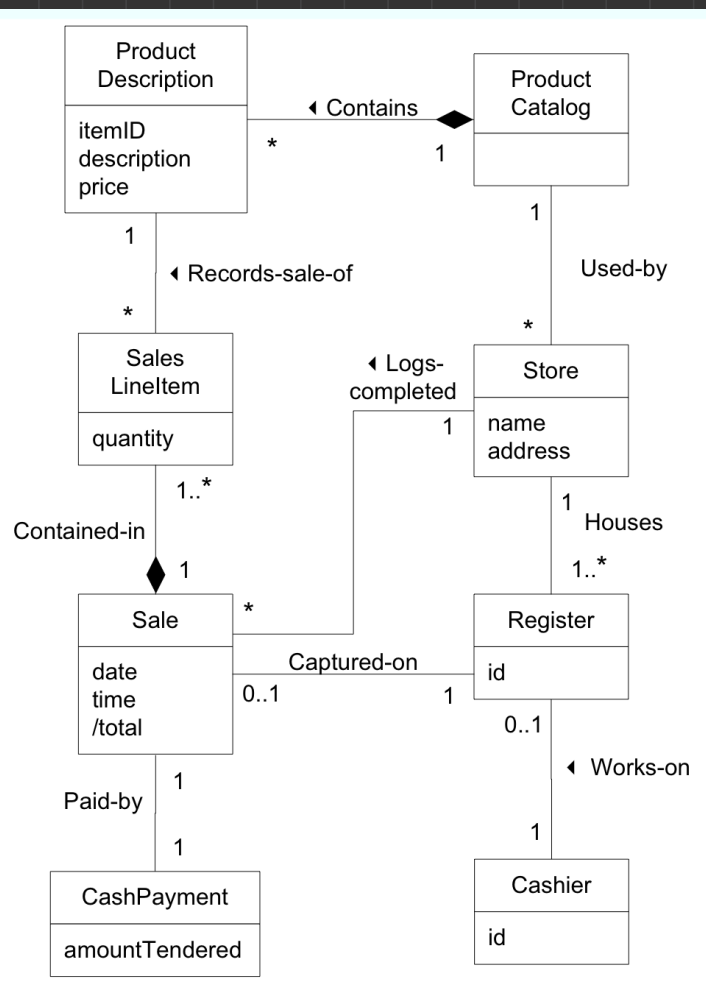


Scenario principale di successo (o flusso di base):

1. Il Cliente arriva alla cassa POS con gli articoli e/o i servizi da acquistare.
2. Il Cassiere inizia una nuova vendita.
3. Il Cassiere inserisce il codice identificativo di un articolo.
4. Il Sistema registra la riga di vendita per l'articolo e mostra la descrizione dell'articolo, il suo prezzo e il totale parziale. Il prezzo è calcolato in base a un insieme di regole di prezzo.

Il Cassiere ripete i passi 3-4 fino a che non indica che ha terminato.

5. Il Sistema mostra il totale, con le imposte calcolate.
6. Il Cassiere riferisce il totale al Cliente, e richiede il pagamento.
7. Il Cliente paga e il Sistema gestisce il pagamento.
8. Il Sistema registra la vendita completata e invia informazioni sulla vendita e sul pagamento ai sistemi esterni di Contabilità (per la contabilità e le commissioni) e di Inventario (per l'aggiornamento dell'inventario).
9. Il Sistema genera la ricevuta.
10. Il Cliente va via con la ricevuta e gli articoli acquistati.

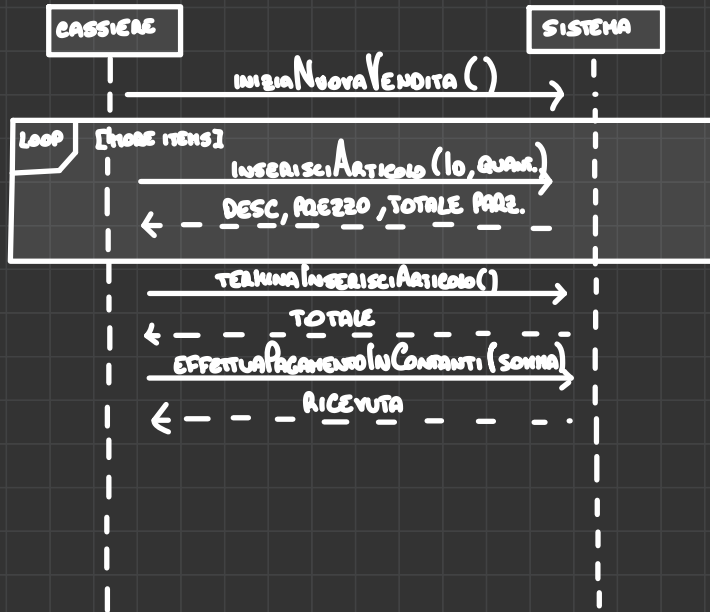


Diagramma di Interazione per *iniziaNuovaVendita*

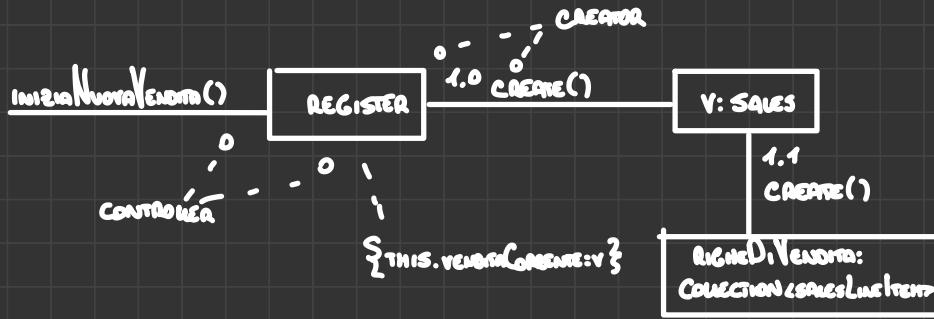
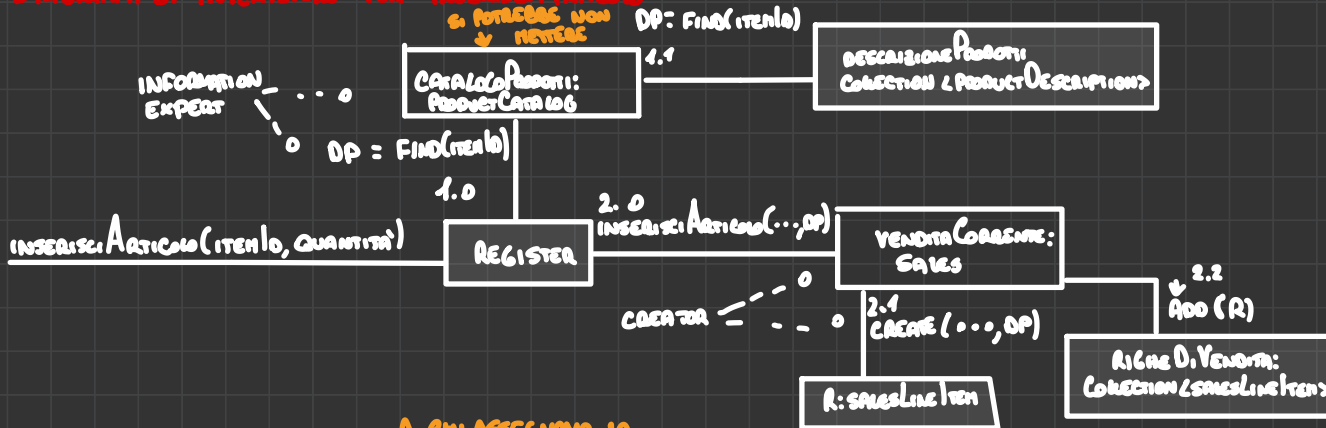


Diagramma di Interazione per *inregistraArticolo*



A CHI ASSEGNARE LA
RESPONSABILITA' DI TENERE
LA PRODUCT DESCRIPTION?

Diagramma di Interazione per Terminare l'Articolo

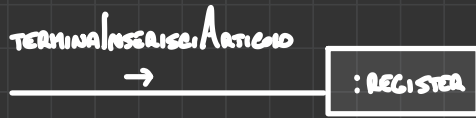


Diagramma di Interazione Effettua Pagamento in Contanti

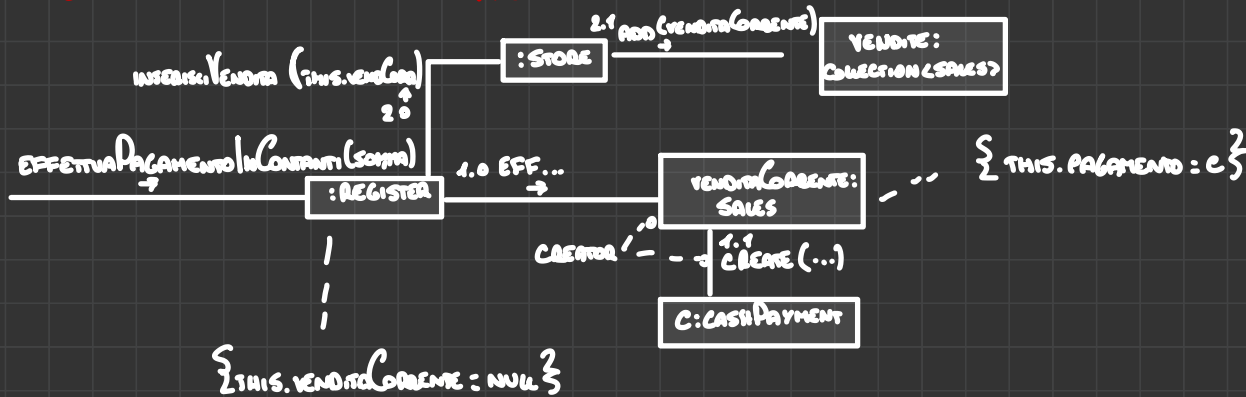
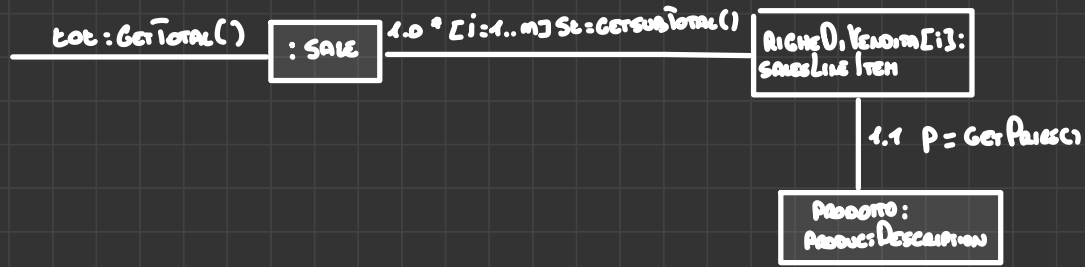


Diagramma di Interazione GetTotal (interrogazione)



Contratto CO2: enterItem

Operazione: enterItem(itemID: ItemID, quantity: integer)

Riferimenti: Caso d'uso: Elabora Vendita

Pre-condizioni: È in corso una vendita s.

Post-condizioni:

- è stata creata un'istanza sli di SalesLineItem
- sli è stata associata con la Sale corrente s
- sli è stato associato con una ProductDescription,
in base alla corrispondenza con itemID
- sli.quantity è diventato quantity

Acne Pizza caso d'uso UC2

Diagramma delle Classi di Progetto

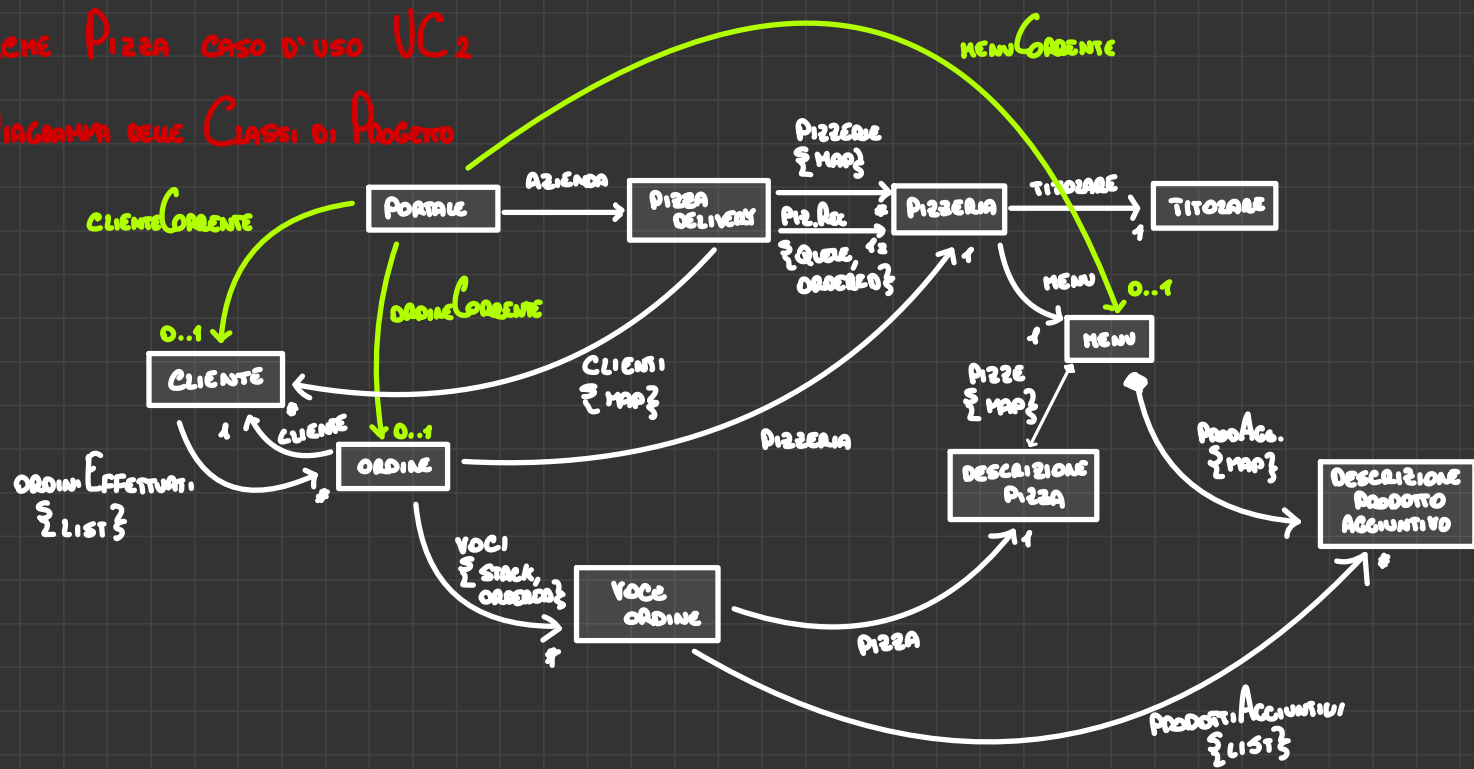


Diagramma di Interazione LOGIN CLIENTE

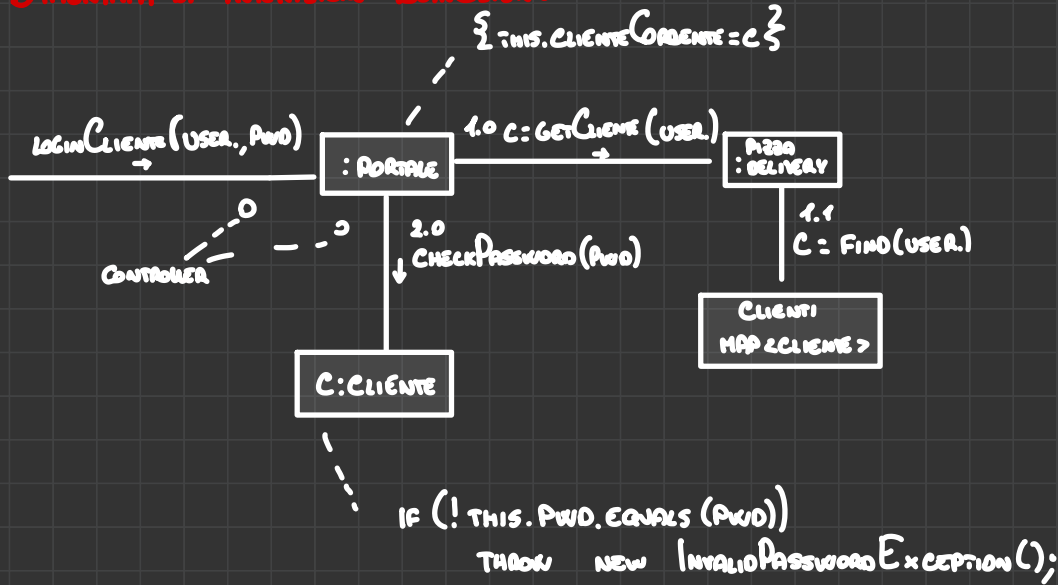


Diagramma di Interazione inizializzazione Ordine

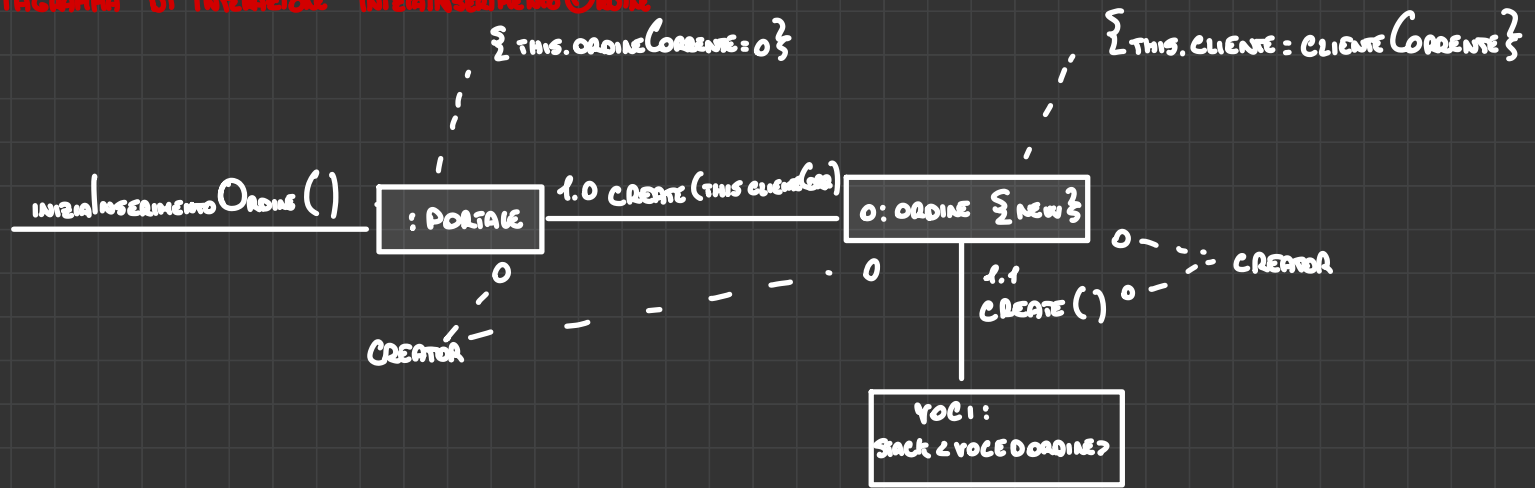


Diagramma di Interazione di SELEZIONACITTA



Diagramma di Interazione di SELEZIONAPIZZERIA

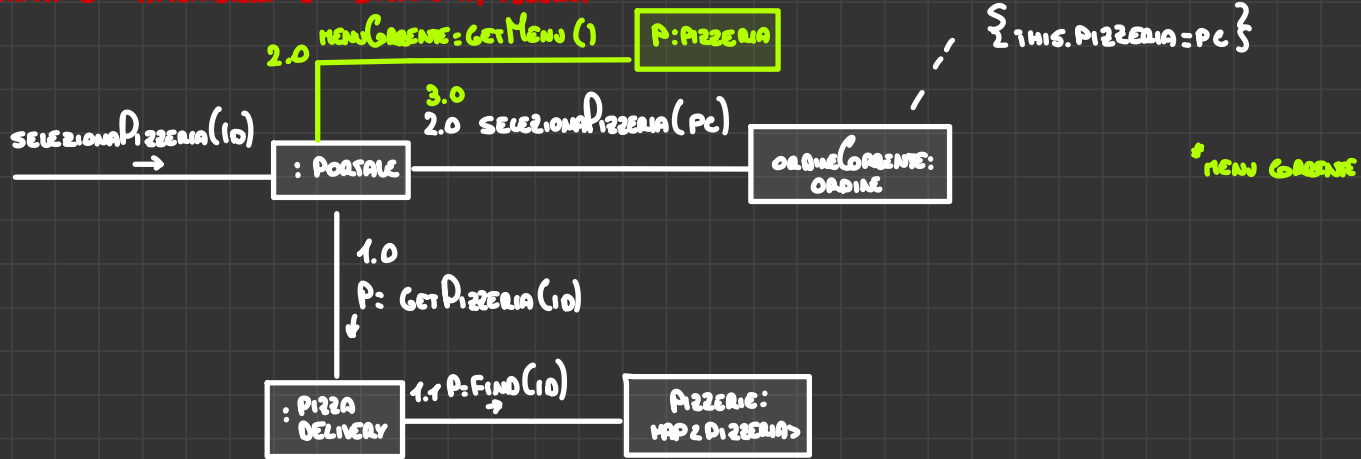


Diagramma di Interazione di Aggiungi Pizza All Ordine

PIZZERIACORRENTE : NON HA SENSO
A STO PUNTO METTO
LA PIZZERIA DI
ORDINE

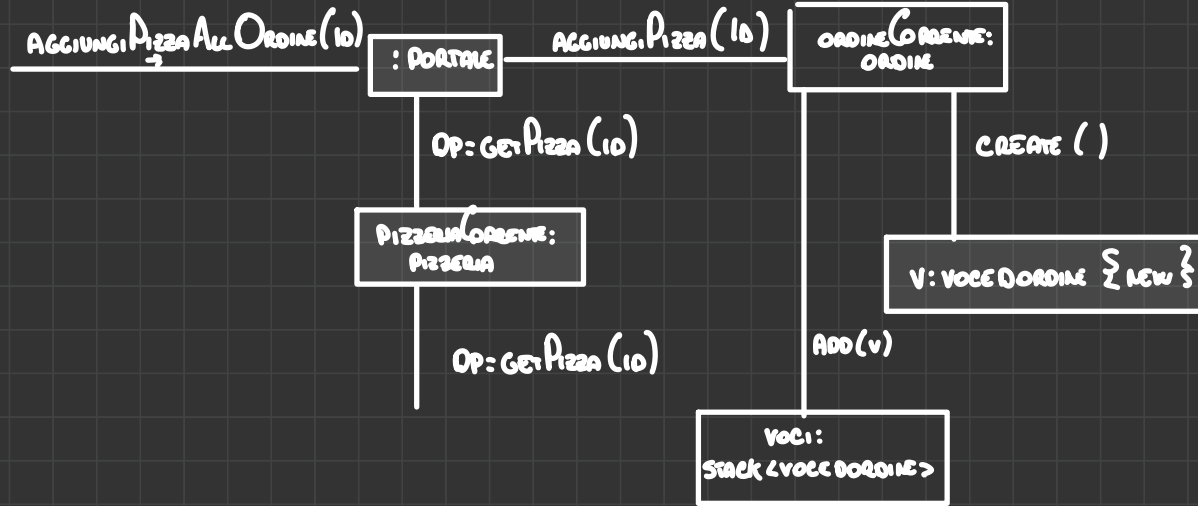


Diagramma di Interazione di Aggiungere Pizza All'Ordine

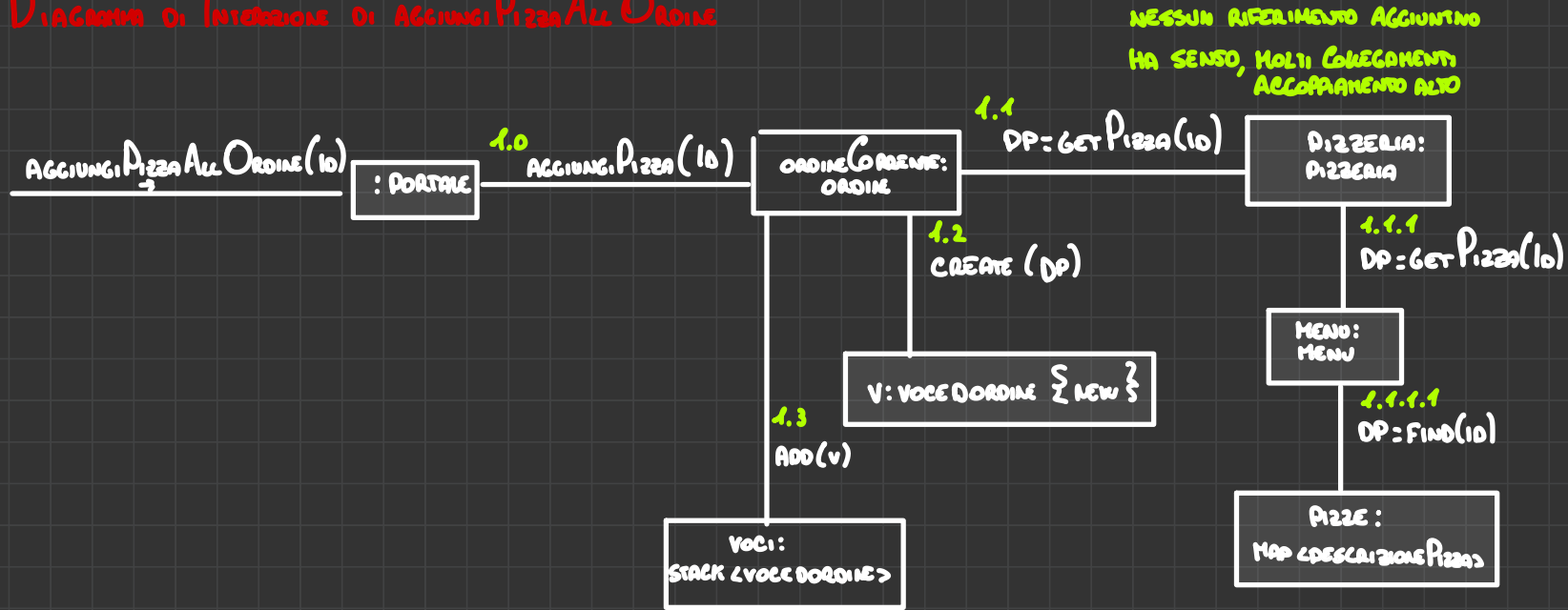


Diagramma di Interazione di Aggiungere Pizza All'Ordine

MENUCORRENTE: BASSO ACCOPPIAMENTO
UN PO' MENO COESIONE

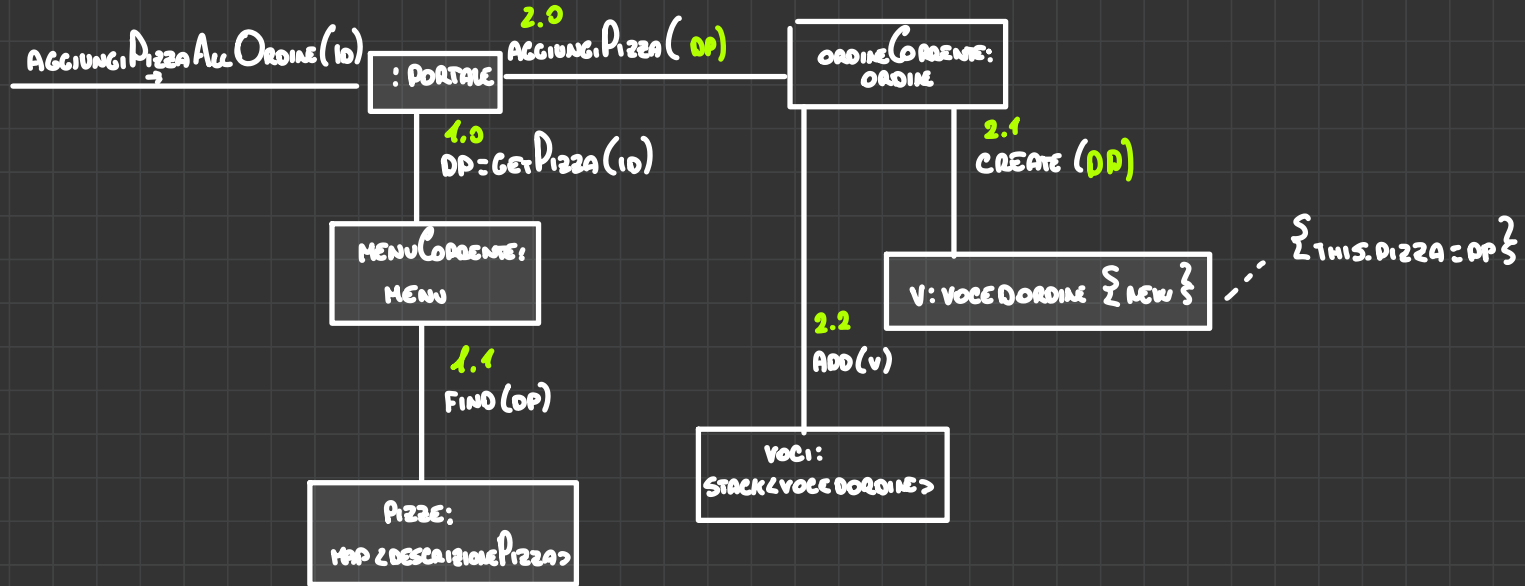


Diagramma di Interazione di Aggiungere Prodotto Aggiuntivo All'ultima P.zza Ordinata

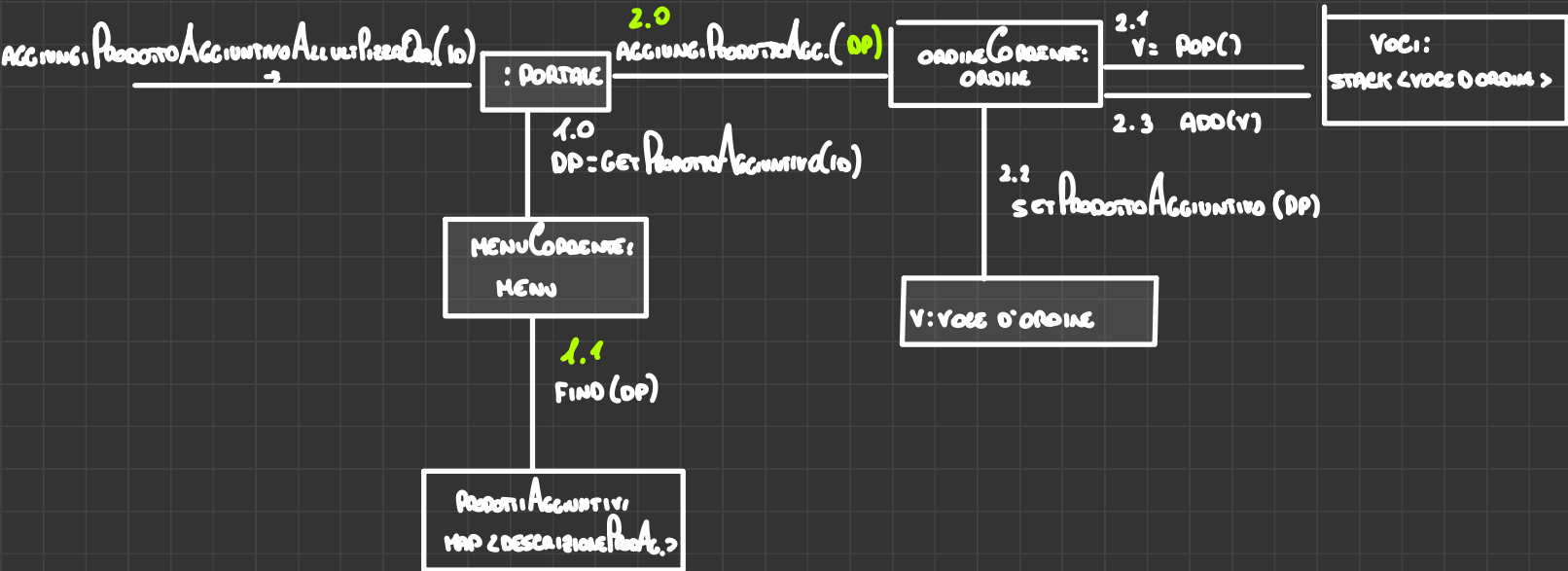


Diagramma di Interazione di Termina Seleziona Prodotto Aggiuntivo

Termina Seleziona Prodotto Aggiuntivo() → : PORTALE

Diagramma di Interazione di Termina Seleziona Pizza

Termina Seleziona Pizza() → : PORTALE

Diagramma di Interazione di Conferma Ordine

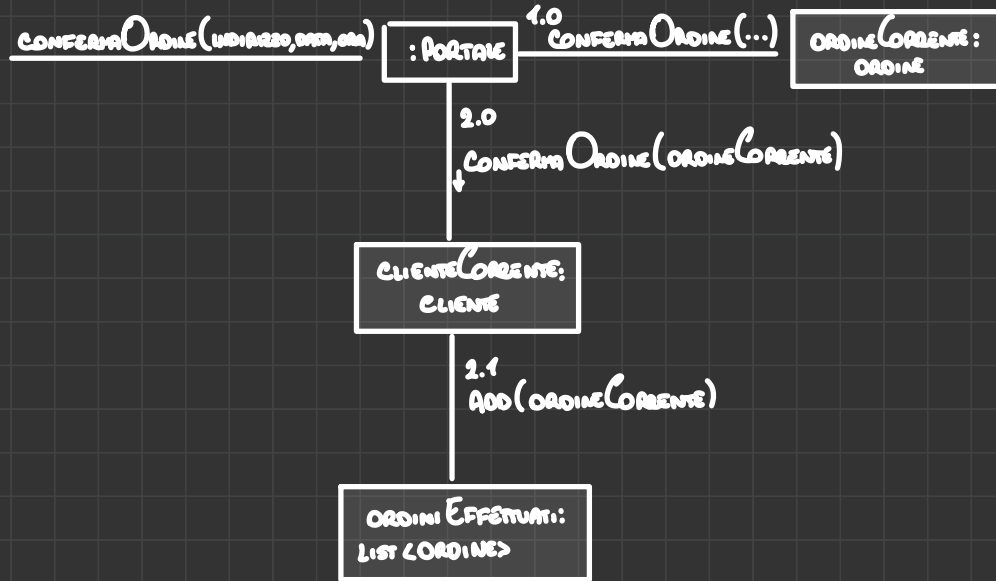
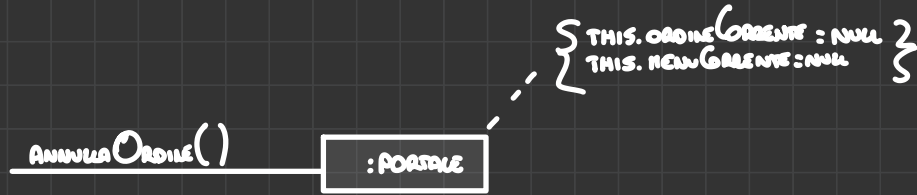


Diagramma di Interazione di Annulla Ordine



RIPASSO ACHE PIZZA

DCD

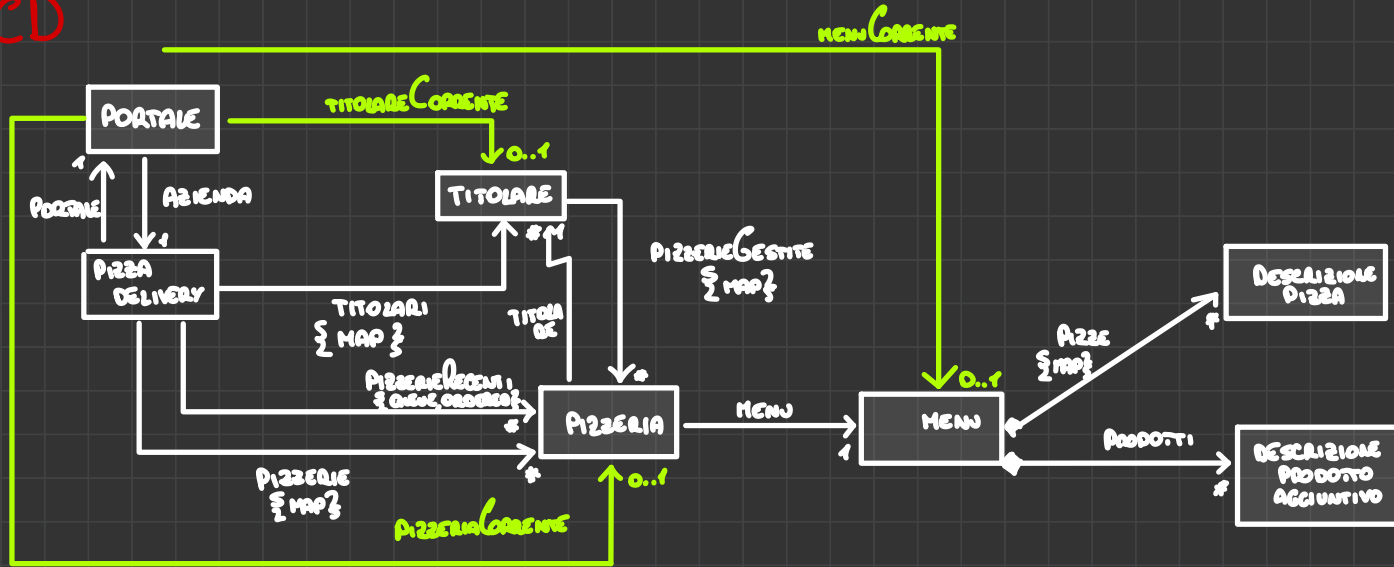


Diagramme d'Interaction LOGIN Titulaire

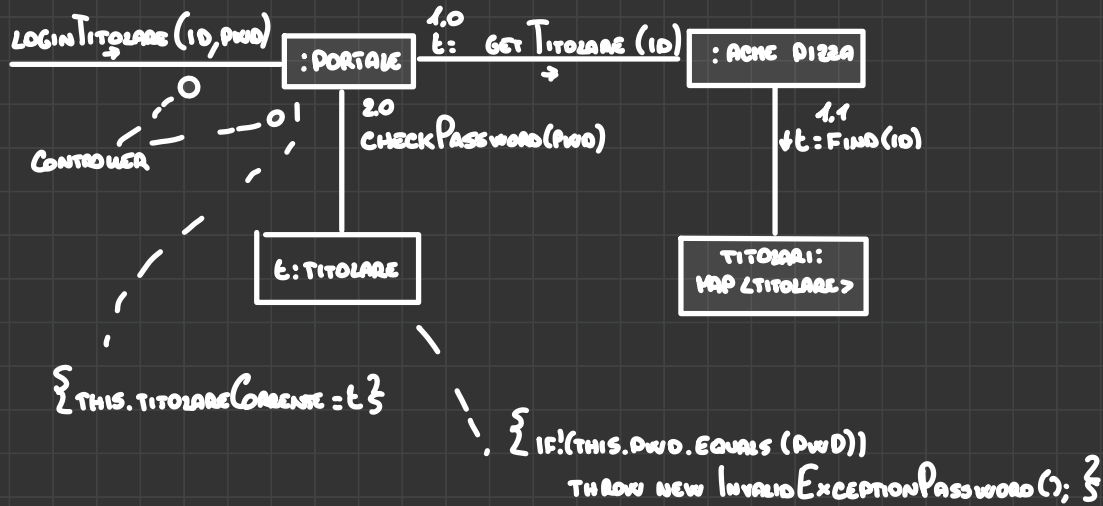


Diagramma di Interazione di Inizializzazione Pizzeria

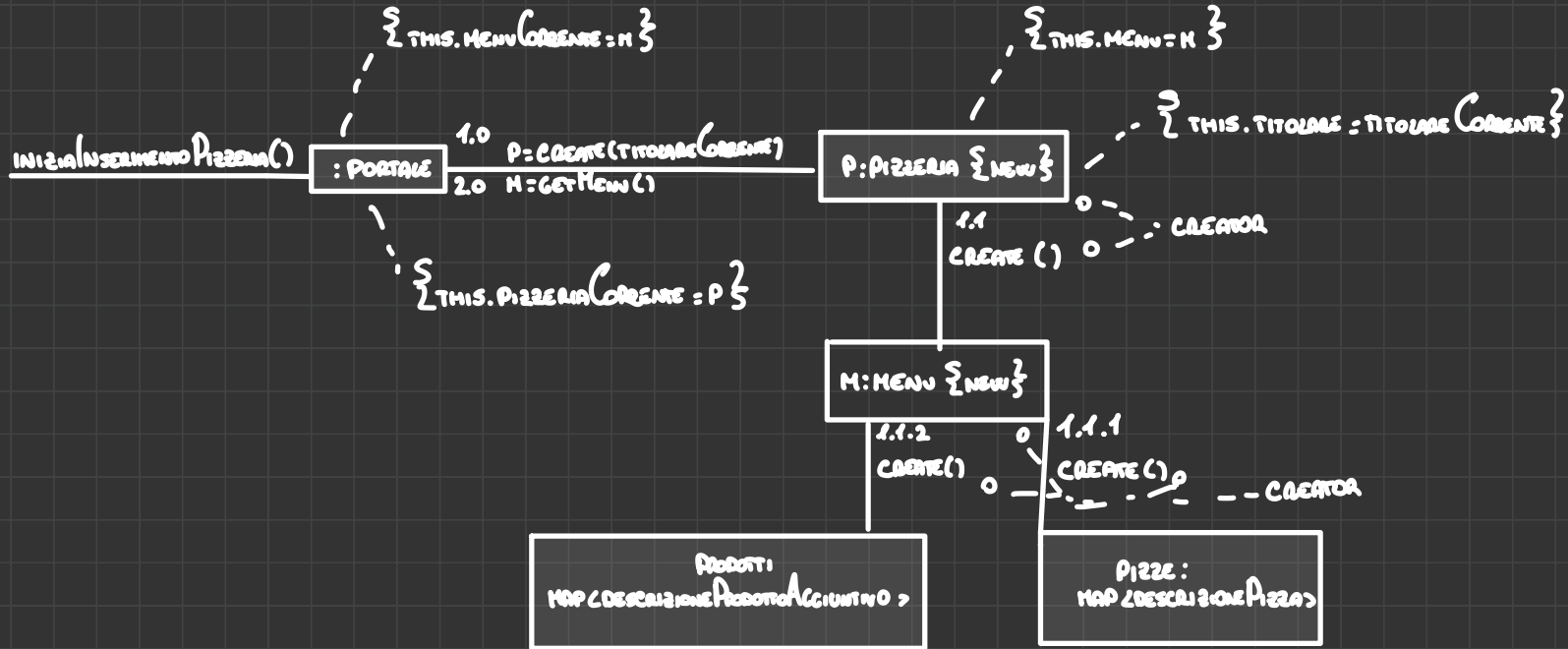


Diagramma di Interazione di Inserisci Dati Pizzeria



Diagramma di Interazione di Inserisci Pizza Nel Menu

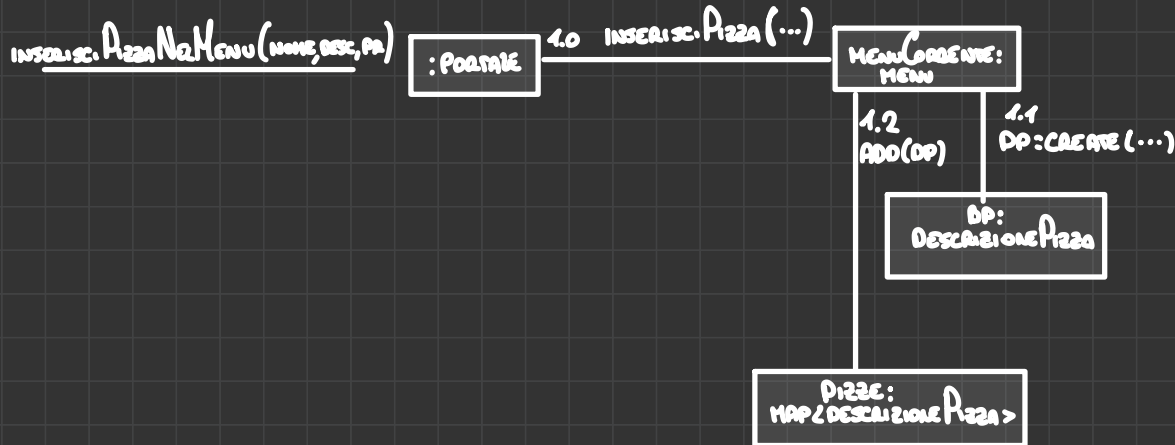


Diagramma di Interazione di Inserisci PA Nel Menu

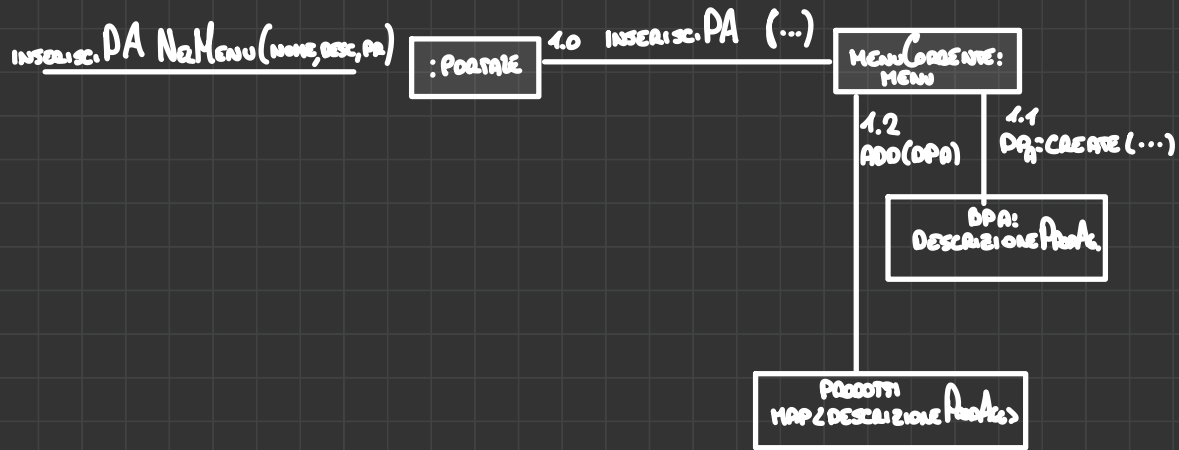


Diagramma di Interazione per Terminare/Inserire Pizze Nel Menu

terminaInserisciPizzaNelMenu() : portale

Diagramma di Interazione per Terminare/Inserire Prodotto Accessorio Nel Menu

terminaProdottoAccessorioNelMenu() : portale

Diagramma di Interazione di conferma/instancia di Pizzeria

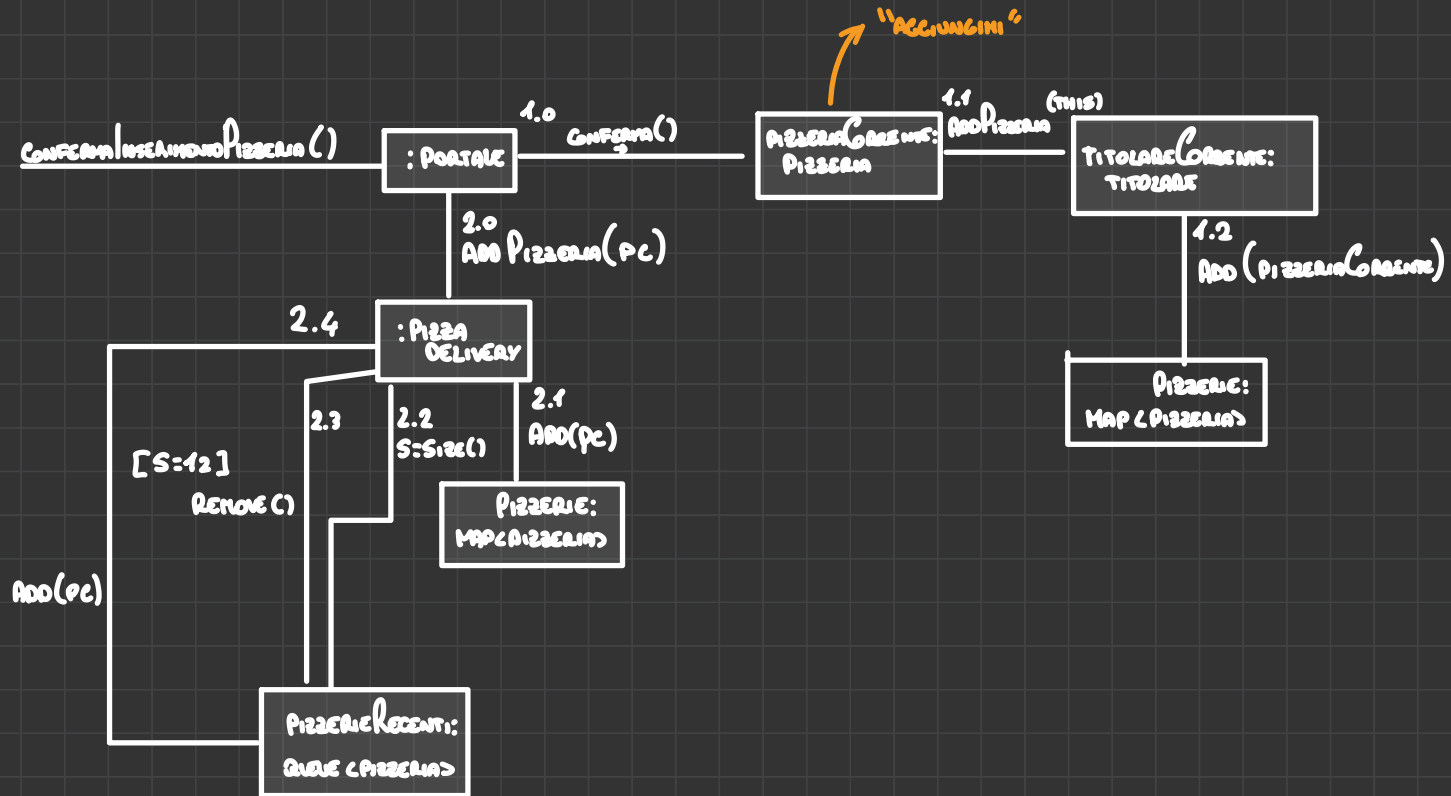
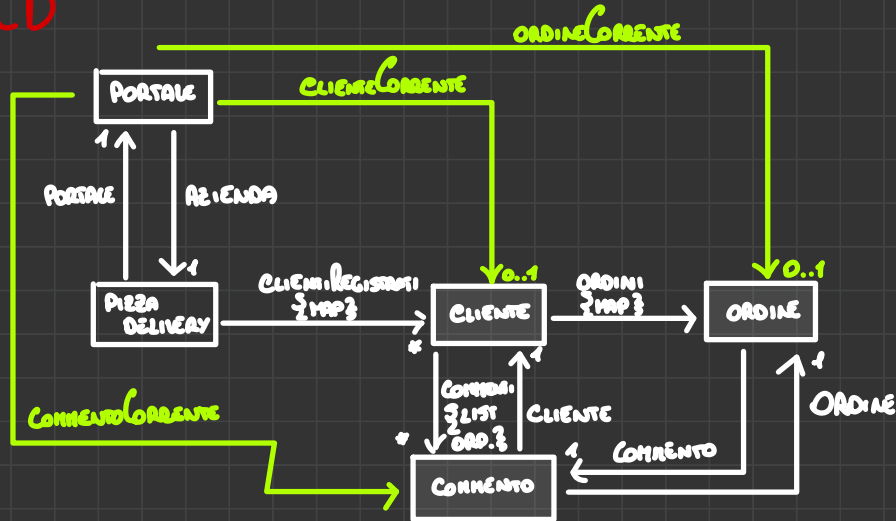


Diagram 0: Interactions 0: AnnualInsuranceProgram

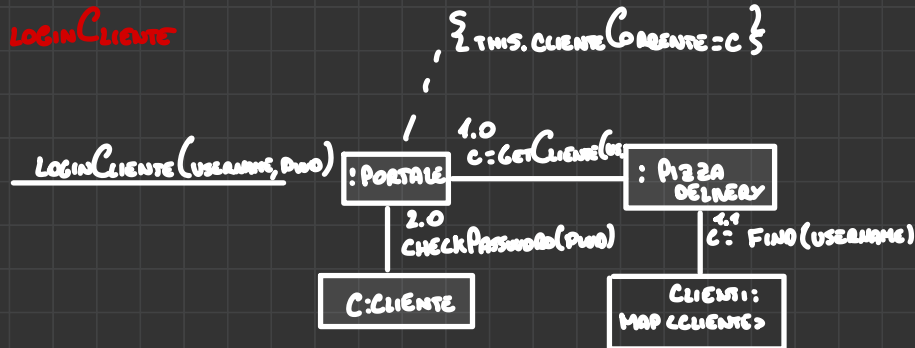
AnnualInsuranceProgram : possible / { this.PossibleCourse = null }

DIAGRAMMI UC10

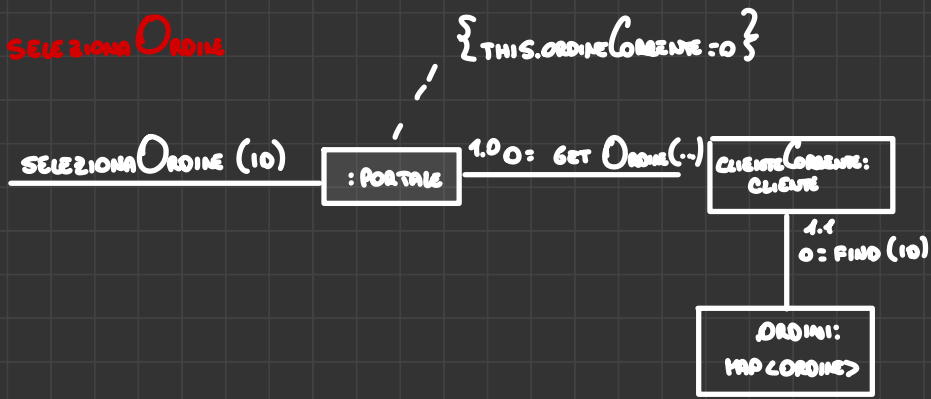
DCD



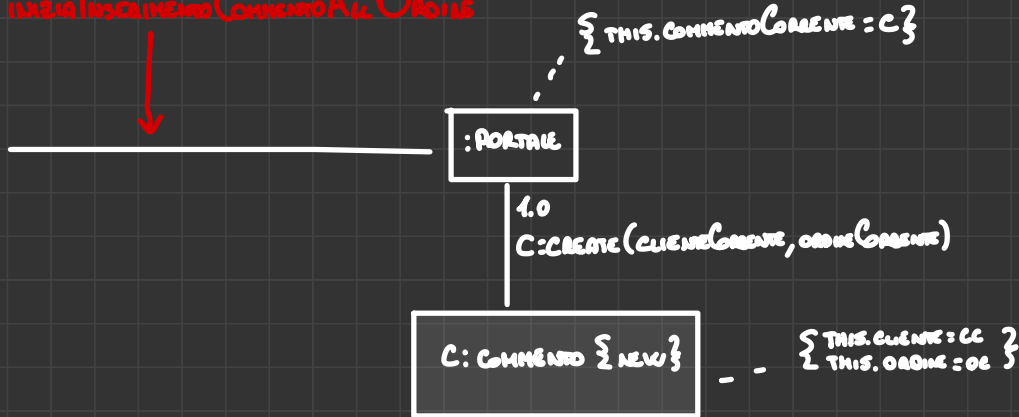
LOGIN CLIENTE



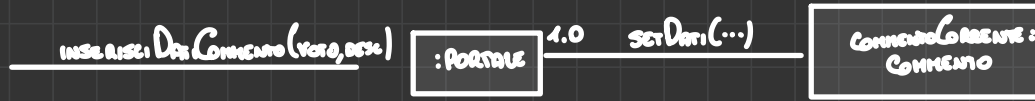
SELEZIONA ORDINE



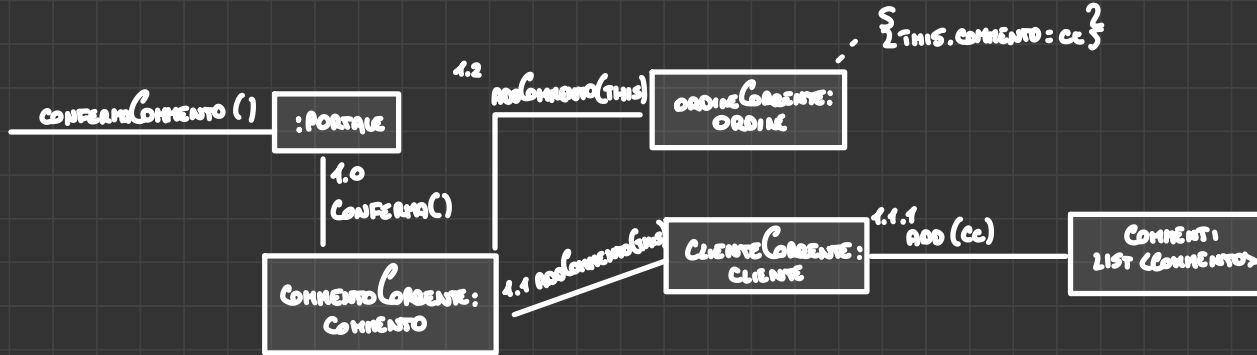
INIZIA INSCRIVENDO COMMENTO ALL ORDINE

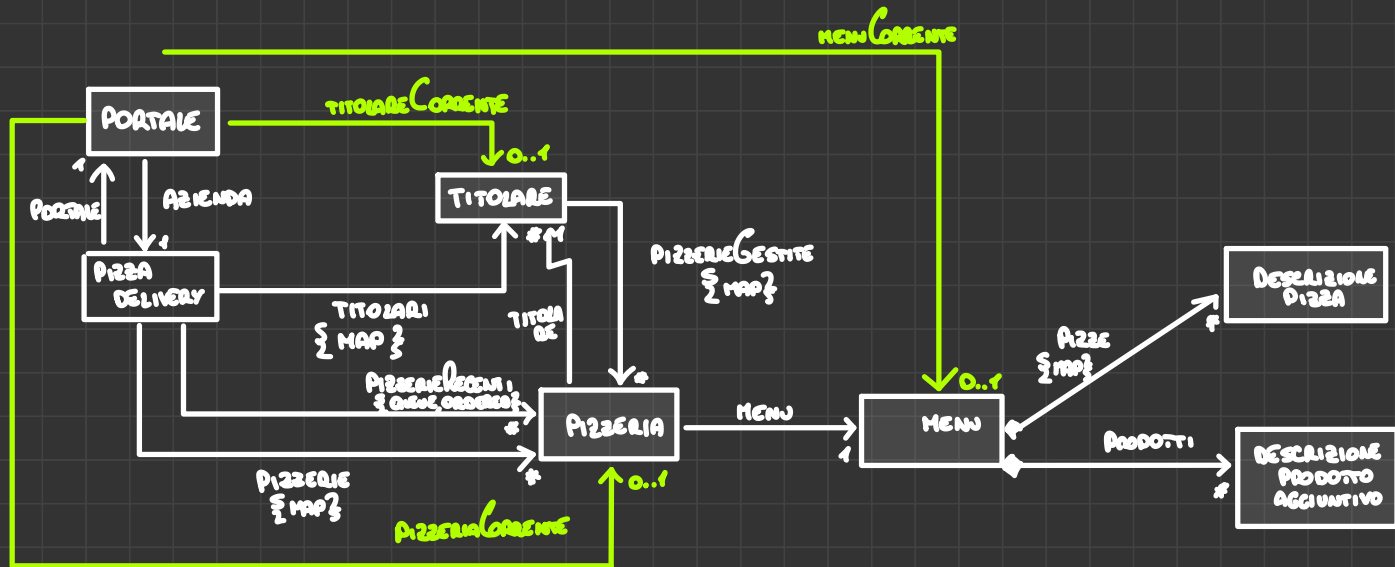


INSERIR: DATA COMMENTO

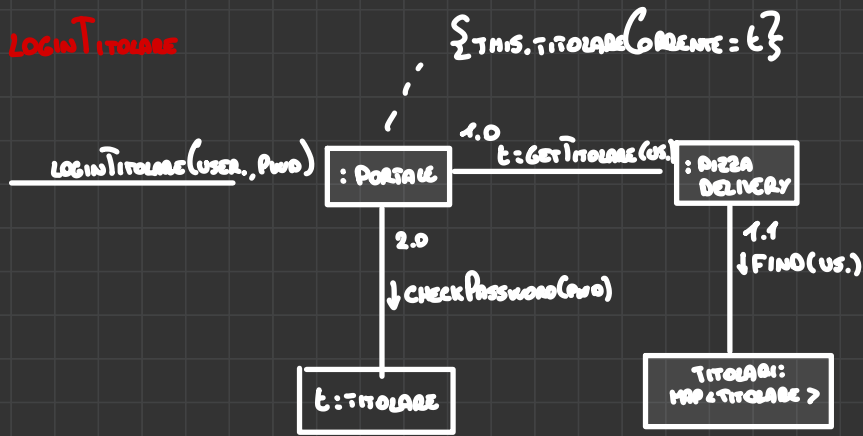


CONFERMA COMMENTO

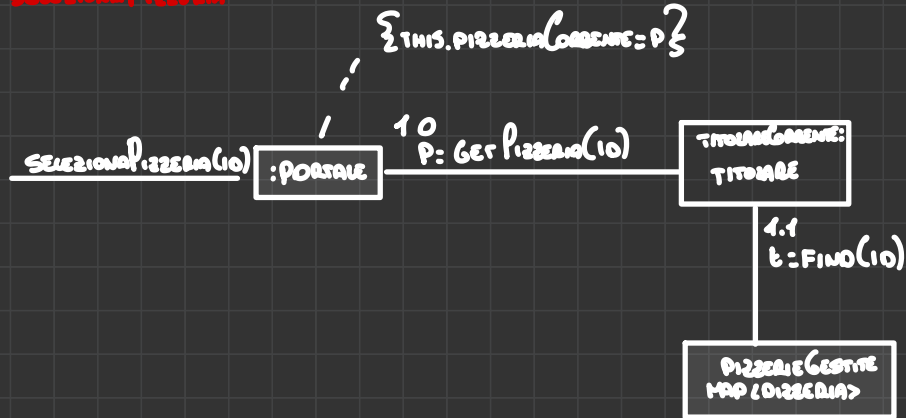




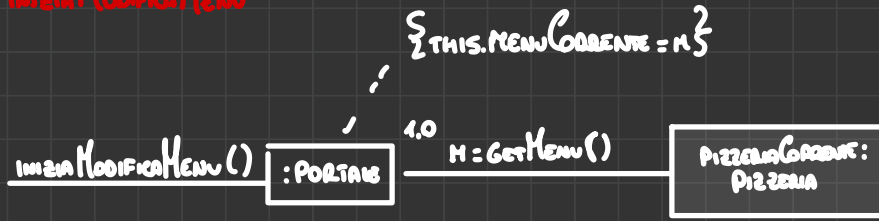
LOGIN T:POLARIS



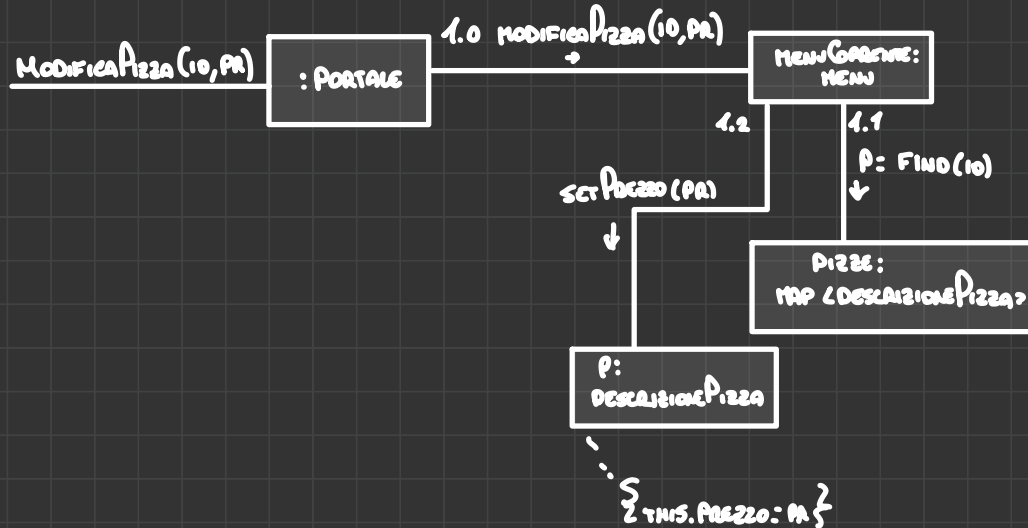
SELECTION P:PIZZERIA



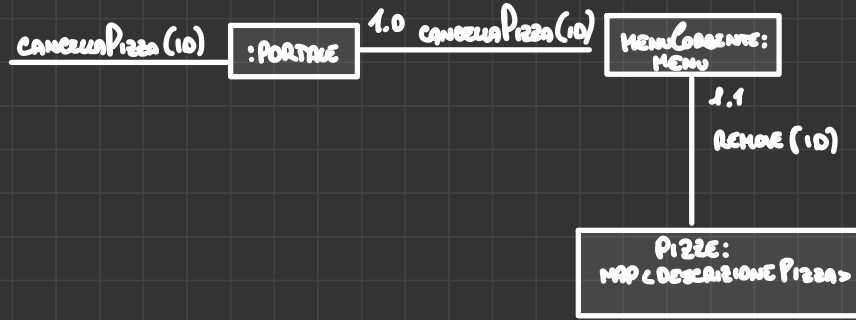
INIZIA MODIFICA MENU



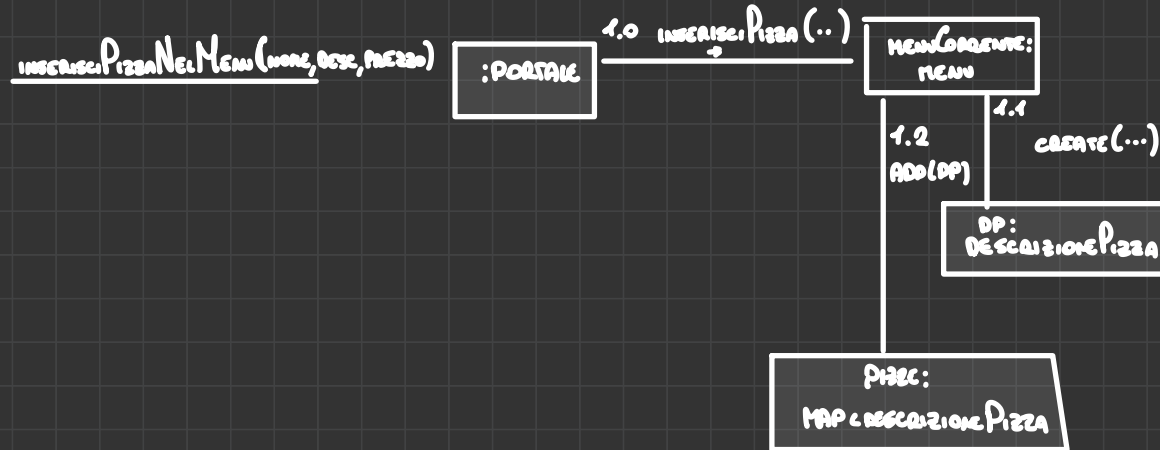
MODIFICA PIZZA



CANCELLA PIZZA



INSECA PIZZA NEL MENU

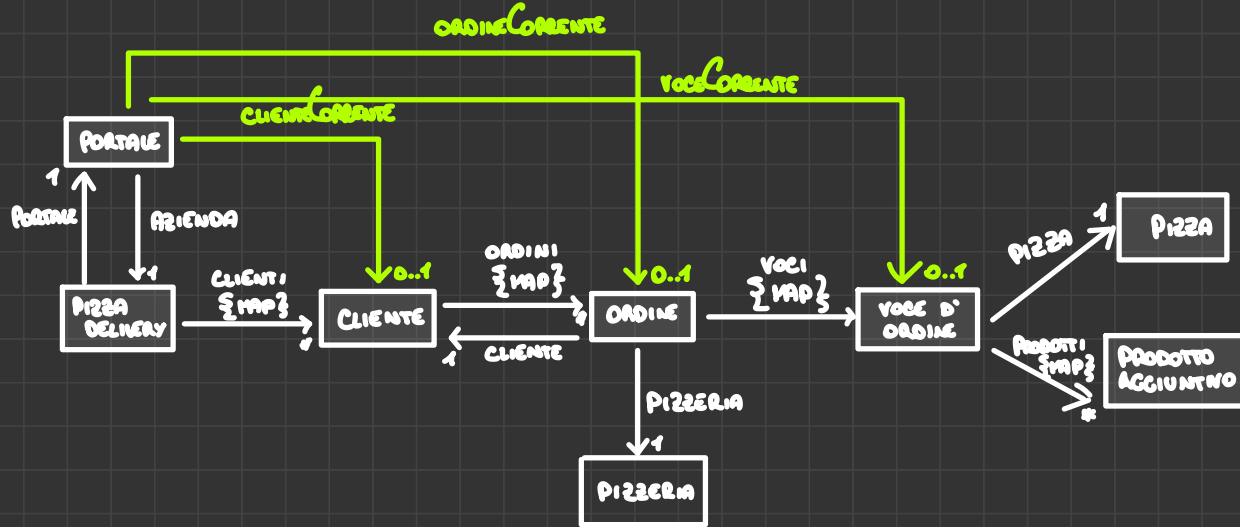


CONFIRM/MODIFICA MENU

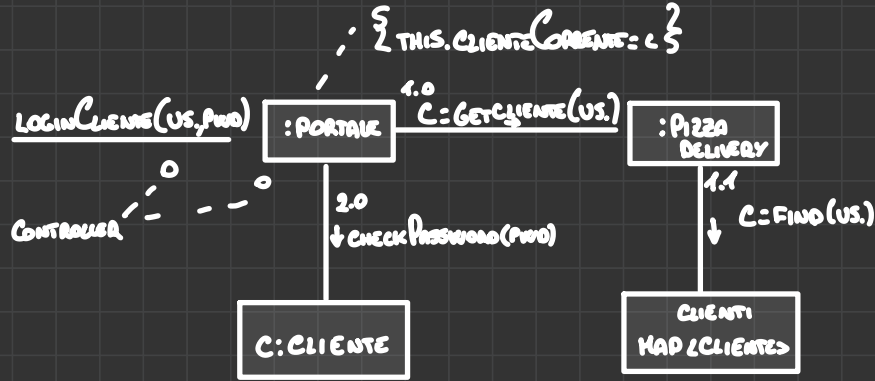
/ { THIS.MENU CORRENTE = NULL }

CONFIRM/MODIFICA MENU() : PORTALE

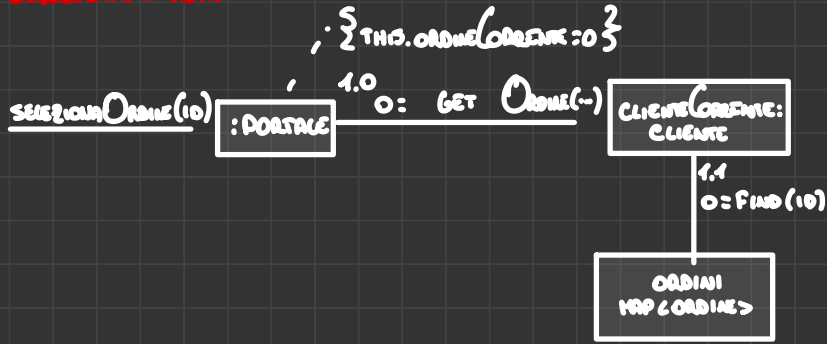
DCD PER MODIFICA ORDINE



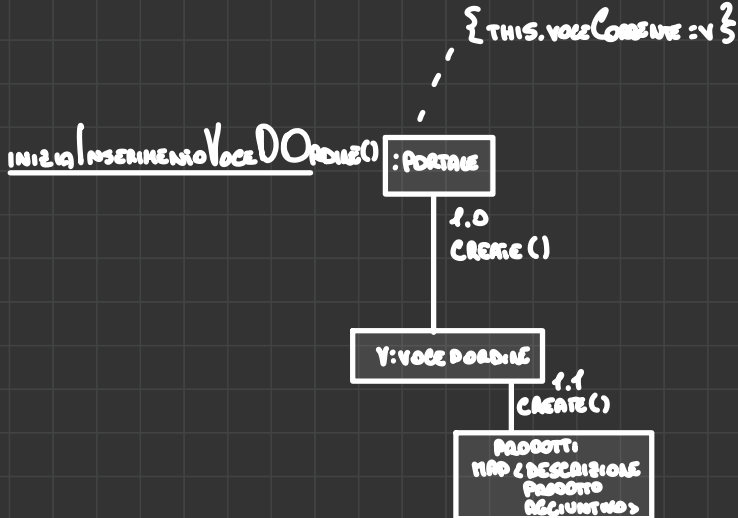
LOGIN CLIENTE



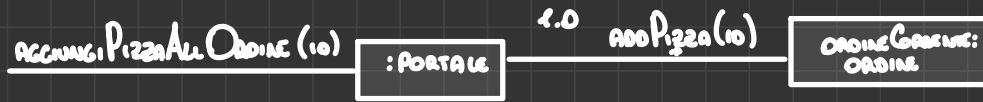
SELEZIONA ORDINE



INIZIA INSERIMENTO VOCE D'ORDINE



adding Pizza All Ordine



Riasso VC2

LOGIN CLIENTE (USE., PASS)

INIZIA INSEALIMENTO ORDINE ()

SELEZIONA CITTA (CITTA)

SELEZIONA PIZZERIA (ID PIZZERIA)

INSERISCI PIZZA NEGL ORDINE (ID PIZZA)

TERMINA INSERISCI PIZZA NEGL ORDINE ()

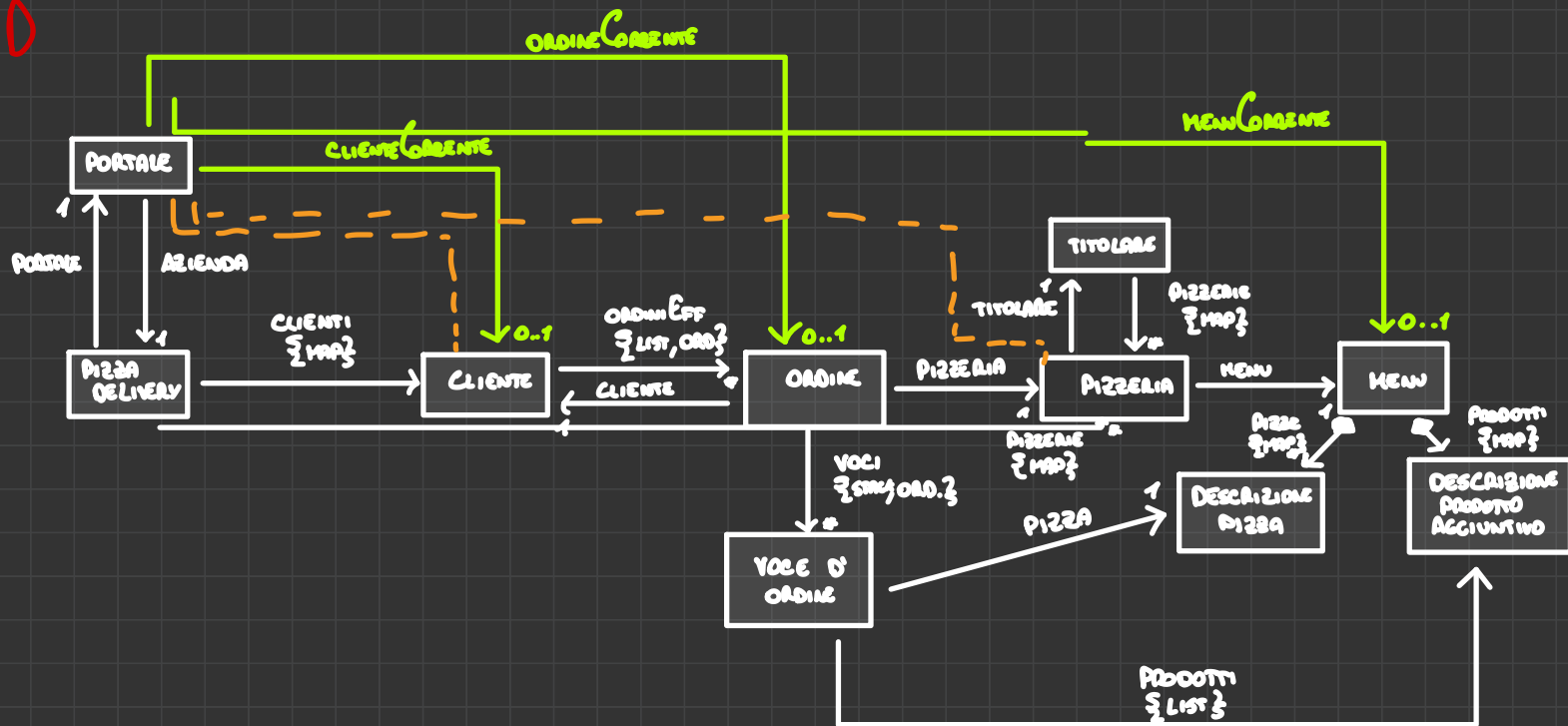
INSERISCI PRODOTTO AGGIUNTIVO NEGL ORDINE (ID PROD)

TERMINA INSERISCI PRODOTTO AGGIUNTIVO NEGL ORDINE ()

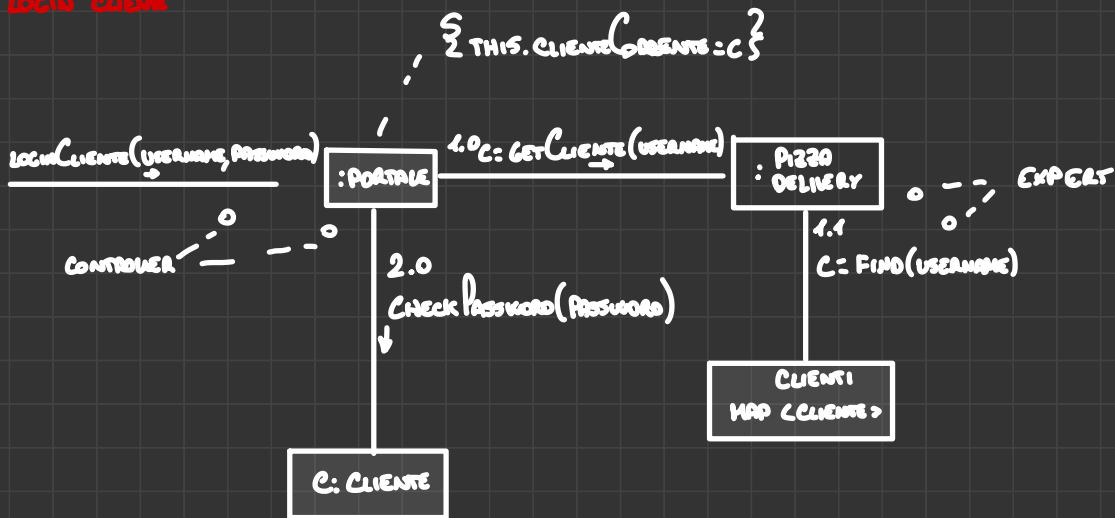
CONFERMA ORDINE (INDIRIZZO, DATA, ORA)

ANNULLA ORDINE ()

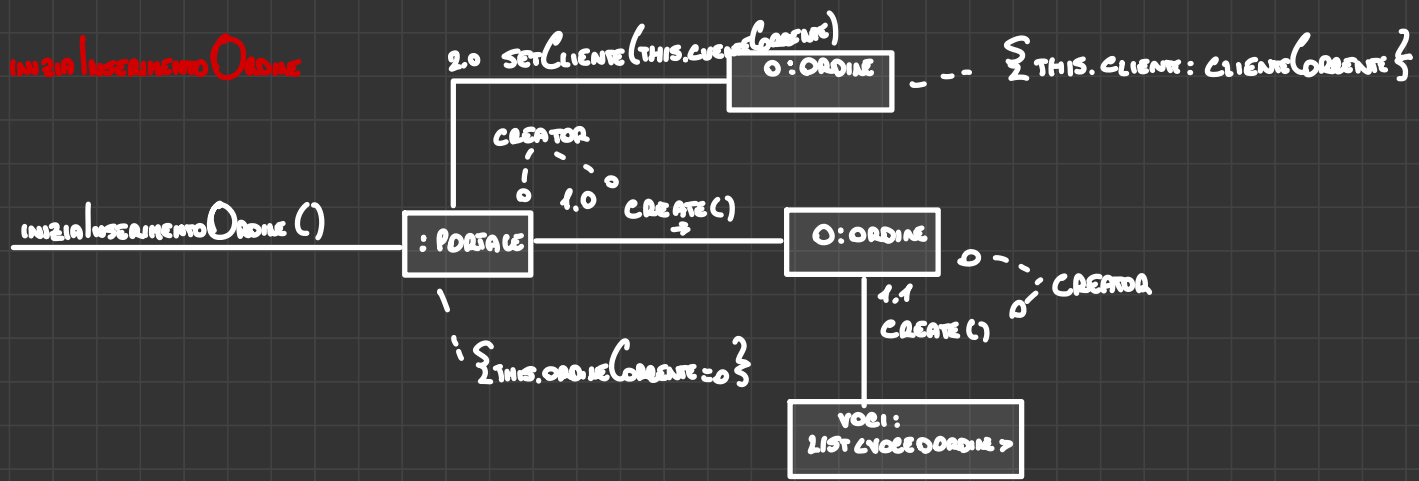
DCD



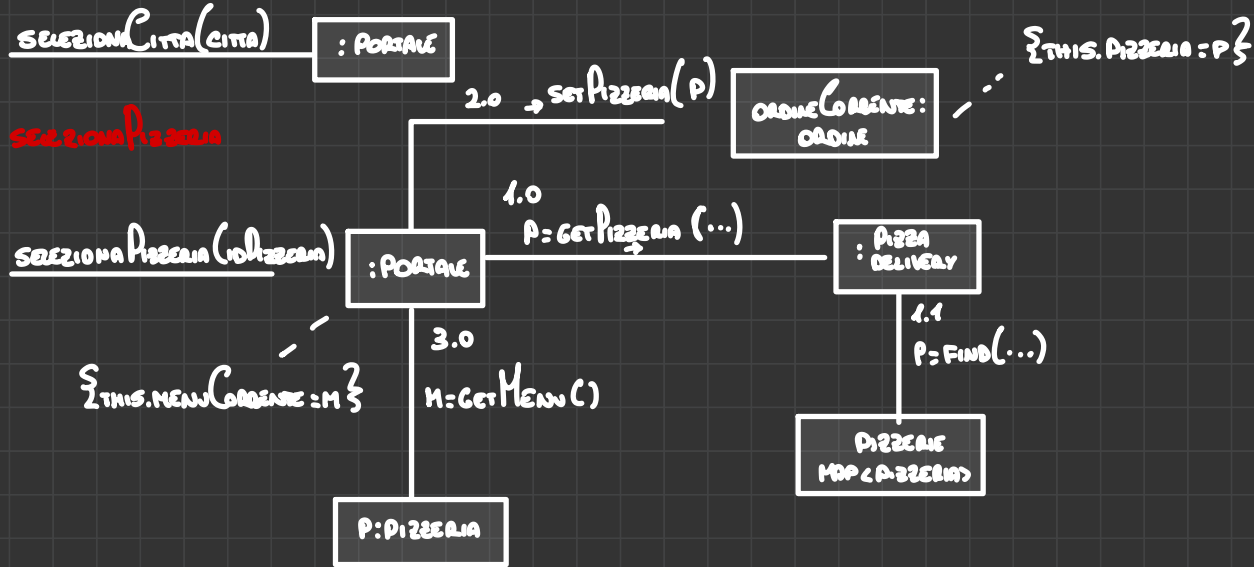
LOGIN CLIENTE



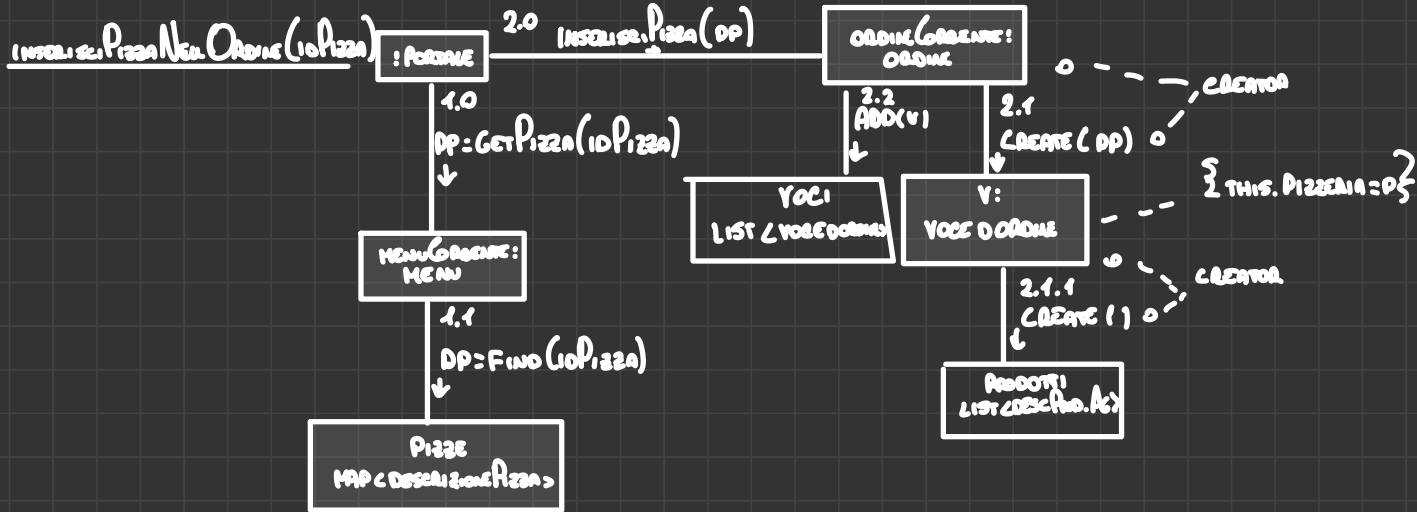
INIZIA INSEGNAMENTO ORDINE



SELEZIONA CITTÀ



INSTRUC: PizzaNewOnline



TERMINA INSTRUCC: PizzaNewOnline

TERMINA INSTRUCC: PizzaNewOnline() : PORTALE

INSEAR: Prodotto Attivo e New Ordine

INSEAR: Prodotto Attivo e New Ordine (10 Pts)

2.0 INSEAR: Prodotto (DPA)
: PORNAME

1.0
DPA: GET Prodotto (10 Pts)
↓

MEM LORENTE:
MEM

1.1
DPA: FIND (10 Pts)
↓

PRODOTTI
MAP < DESCRIZIONE
Prodotto
Attivo >

2.2 SET Prodotto (DPA)
ORDINE LORENTE:
ORDINE

2.1
V: POP()

VOCE:
STACK < VOCE ORDINE >

V: VOCE
ORDINE

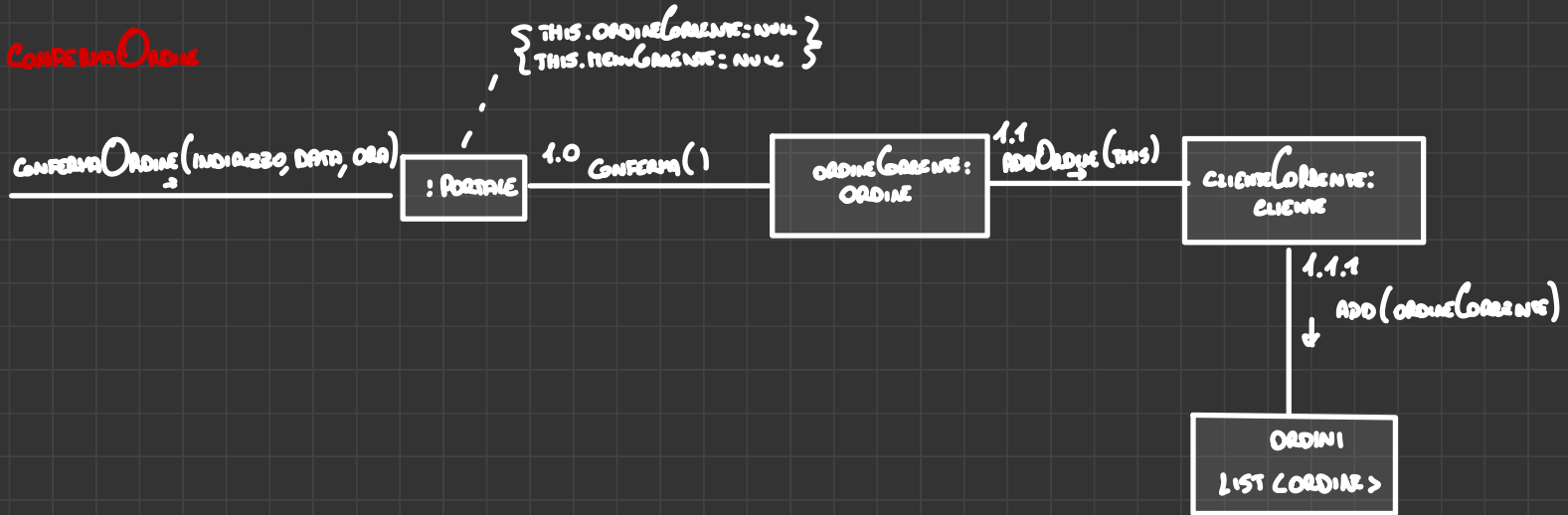
2.2.1
ADD (DPA)

PRODOTTI
LIST < DESCRIZIONE
Prodotto Attivo >

TECHNICAL DESIGN: PORTAL ACCOUNT TO NEW ORDERS

TECHNICAL DESIGN: PORTAL ACCOUNT TO NEW ORDERS (?) : PORTAL

CONFIRM ORDERS



ANNULE ORDINE

ANNULE ORDINE ()

: PORTALE

{ THIS. ORDINECORRENTE = null
THIS. MENUCORRENTE = null }

CODICE RELATIVO A INSERISCI PIZZA NEU ORDINE

PUBLIC CLASS PORTALE {

PRIVATE MENU MENUCORRENTE;

PRIVATE ORDINE ORDINECORRENTE;

PRIVATE VOID INSERISCIPIZZANEUORDINE (LONG IDPIZZA) {

DESCRIZIONEPIZZA OP = THIS.MENUCORRENTE.GETPIZZA(IDPIZZA);

THIS.ORDINECORRENTE.INSERISCIPIZZA(OP);

}

}

```
PUBLIC CLASS MENU {
```

```
    PRIVATE MAP<LONG, DESCRIZIONEPIZZA> PIZZA;
```

```
    PRIVATE DESCRIZIONEPIZZA GETPIZZA(LONG IDPIZZA) {
```

```
        RETURN THIS.PIZZA.GET(IDPIZZA);
```

```
    }
```

```
}
```

```
PUBLIC CLASS ORDINE {
```

```
    PRIVATE LIST<VOCEORDINE> VOCEI;
```

```
    PRIVATE VOID INSERISCIPIZZA (DESCRIZIONEPIZZA DP) {
```

```
        VOCEORDINE V = NEW VOCEORDINE (DP)
```

```
        THIS.VOCEI.ADD(V)
```

```
    }
```

```
}
```

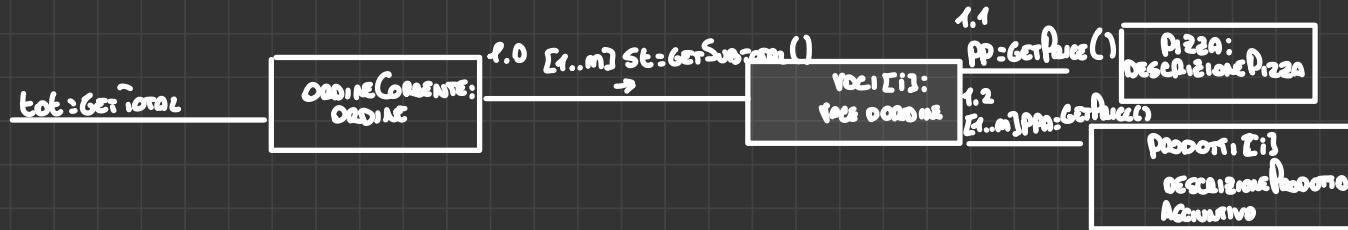
```

PUBLIC CLASS VociOrdine {
    PRIVATE DESCRIZIONEPIZZA pizza;
    PRIVATE MAP<Long, DESCRIZIONE PRODOTTO ACCUMULATO> prodotti;

    PUBLIC VociOrdine (DESCRIZIONEPIZZA dp) {
        THIS.PRODOTTI = new HashMap<>();
        THIS.PIZZA = dp;
    }
}

```

GET TOTAL



AcmePayroll (nel seguito chiamato solo “sistema”) è il sistema client-server usato dal **Laboratorio Acme** (it.wikipedia.org/wiki/Laboratorio_ACME, nel seguito “Acme”) per il calcolo delle buste paga dei suoi dipendenti.

Il sistema deve registrare opportunamente le **presenze dei dipendenti**, memorizzando per ciascuna presenza la **data**, **l'ora di entrata e l'ora di uscita**. (Caso d'uso UC1.)

La Acme produce **marchingegni**. Ciascuna **tipologia di marchingegno** ha un **nome e un prezzo di vendita** (per esempio, un *Pianoforte esplosivo* costa 5000 €). Ogni volta che un dipendente (o un gruppo di dipendenti) della Acme completa la **costruzione di un marchingegno**, il sistema deve registrare opportunamente il **marchingegno costruito**, memorizzando anche la **data di completamento e il dipendente** (o il gruppo di dipendenti) che lo ha costruito. (Caso d'uso UC2.)

Per semplicità, si supponga che il sistema debba memorizzare solo le presenze dei dipendenti dell'ultimo mese, e solo i dati sui marchingegni completati nell'ultimo mese.

Alla fine del mese, il sistema deve calcolare quanto deve essere pagato ciascuno dei dipendenti. (Caso d'uso UC4.) I dipendenti vengono pagati in base alle loro presenze presso la Acme e a quanti e quali marchingegni hanno costruito. Per la precisione, ciascun dipendente si accorda con la Acme per il pagamento della propria attività in base ad alcuni tipi di contratti predefiniti. I contratti di interesse in questa iterazione sono:

- **Contratto con pagamento forfettario**: In un mese, il dipendente viene pagato un **certo importo fisso**, indipendentemente dalle sue presenze e dal numero di marchingegni che ha costruito in quel mese. L'importo fisso può variare da dipendente a dipendente, ed è scritto nel contratto del dipendente. Per esempio, 2000 € al mese.
- **Contratto con pagamento a giorni di presenza**: In un mese, il dipendente viene pagato in base al **numero di giorni in cui è stato effettivamente presente in quel mese**, indipendentemente dal numero di marchingegni che ha costruito e dal numero di ore di presenza effettiva. L'importo pagato per ciascun giorno di presenza può variare da dipendente a dipendente, ed è scritto nel contratto del dipendente. Per esempio, **250 € a giorno di presenza**.
- **Contratto con pagamento a ore di presenza**: In un mese, il dipendente viene pagato in base al numero di ore in cui è stato effettivamente presente in quel mese, indipendentemente dal numero di marchingegni che ha costruito e dal numero di giornate di presenza. L'importo pagato per ciascuna ora di presenza può variare da dipendente a dipendente, ed è scritto nel contratto del dipendente. Per esempio, **30 € a ora di presenza**.
- **Contratto con pagamento per marchingegni**: In un mese, il dipendente viene pagato in base ai marchingegni che ha costruito in quel mese, indipendentemente dalle sue presenze in quel mese. L'importo pagato per ciascun marchingegno costruito va calcolato come segue: La **quota per dipendente di un marchingegno (QDM)** è il prezzo di vendita del marchingegno diviso il numero di dipendenti che ha partecipato alla sua costruzione. In corrispondenza, l'importo da pagare al dipendente è la QDM moltiplicata per una **percentuale**, che può variare da dipendente a dipendente, che è scritta nel contratto del dipendente. Per esempio, il 10% della QDM. Per esempio: se un dipendente ha partecipato alla costruzione di un *Pianoforte esplosivo*, il cui prezzo è 5000 €, e questo è stato fatto da un gruppo di 5 dipendenti, allora la QDM è 1000 €; se il dipendente viene pagato con una percentuale del 10% della QDM, allora per la costruzione di quel marchingegno gli vanno pagati 100 €.

Ogni dipendente si accorda con la Acme per un **contratto mensile** (questo di solito avviene entro il giorno 15 di ciascun mese). Il contratto mensile di un dipendente può cambiare di mese in mese. (Caso d'uso UC3.) Il sistema deve memorizzare tutti i contratti dei dipendenti, sia quelli **relativi al mese corrente che ai mesi precedenti**.

L'uso del sistema in discussione è descritto principalmente dai seguenti casi d'uso, descritti in modo sintetico:

Caso d'uso UC1: Gestione presenza – Attore primario: un Dipendente.

Il Dipendente usa il Sistema per registrare la propria entrata alla Acme o la propria uscita dalla Acme. Il Sistema registra (o aggiorna) la presenza.

Caso d'uso UC2: Completamento marchingegno – Attore primario: un Dipendente.

Il Dipendente usa il Sistema per registrare il completamento di un marchingegno. Il Dipendente indica il tipo di marchingegno costruito e inserisce il dipendente o i dipendenti che hanno costruito il marchingegno. Il Sistema registra il marchingegno.

Caso d'uso UC3: Contratto dipendente – Attore primario: l'Ufficio del Personale.

L'Ufficio del Personale usa il Sistema per registrare, per un certo Dipendente, il contratto da utilizzare per quel Dipendente in quel mese, con i relativi parametri.

Caso d'uso UC4: Pagamento dipendente – Attore primario: l'Ufficio del Personale.

L'Ufficio del Personale usa il Sistema per calcolare, per un certo Dipendente, l'importo che va pagato a quel Dipendente per il lavoro svolto nell'ultimo mese.

Analisi e progettazione del software

Studio di caso: Acme Payroll

Ipotesi di lavoro, valide per tutti gli esercizi di progettazione.

- In tutti gli esercizi che seguono, si faccia l'ipotesi che il sistema in discussione gestisca i propri dati solo in memoria principale. Si supponga anche che durante il caso d'uso di avviamento vengano creati e caricati in memoria tutti gli oggetti le cui informazioni siano già effettivamente disponibili al momento dell'avviamento.
- Per ciascuna operazione di sistema va creato un diagramma di interazione che descrive l'interazione relativa alla trasformazione (cambiamento di stato) provocata dall'operazione di sistema. Per quanto riguarda invece le risposte (interrogazioni) restituite dal sistema, se nessun esercizio lo richiede esplicitamente allora non bisogna mostrare nei diagrammi di interazione né il calcolo dei dati da restituire né la loro visualizzazione. Tuttavia, per le risposte del sistema, è comunque necessario verificare che i dati da restituire possano essere (facilmente) calcolati sulla base delle navigabilità tra gli oggetti che sono state progettate (vedi anche il punto successivo).
- **Le soluzioni individuate dovranno essere compatibili (in particolare in termini di visibilità, ovvero di navigabilità delle associazioni) con la realizzazione di TUTTE le operazioni di TUTTI i casi d'uso descritti tra i requisiti.**
- **Nei diagrammi di interazione, mostrare IN MODO ESPLICITO: tutti gli OGGETTI che partecipano all'interazione (compresi gli OGGETTI COLLEZIONE), tutti i MESSAGGI scambiati tra oggetti, tutte le CREAZIONI di oggetti e tutte le FORMAZIONI e ROTTURE di COLLEGAMENTI.**
- Nei diagrammi di interazione, motivare le scelte di progetto fatte indicando i pattern GRASP e GoF applicati.
- Nei diagrammi delle classi di progetto, mostrare: (1) per ciascuna classe: il nome della classe, i nomi dei suoi attributi, i nomi delle sue operazioni; e (2) per ciascuna associazione e ciascuna sua estremità navigabile: la freccia di navigabilità, il nome dell'estremità, la molteplicità e, in caso di associazione navigabile a molti, il tipo di collezione scelta.

Esercizio A1 (Modellazione di dominio)

Fare l'analisi orientata agli oggetti per il sistema in discussione, come segue:

- Mostrare il modello di dominio, relativo a TUTTI i casi d'uso e a TUTTE le regole di dominio.

Esercizio A2 (Progettazione)

Considerando per questo esercizio tutti i tipi di contratti per il pagamento dei dipendenti, fare la progettazione orientata agli oggetti per il sistema in discussione, come segue:

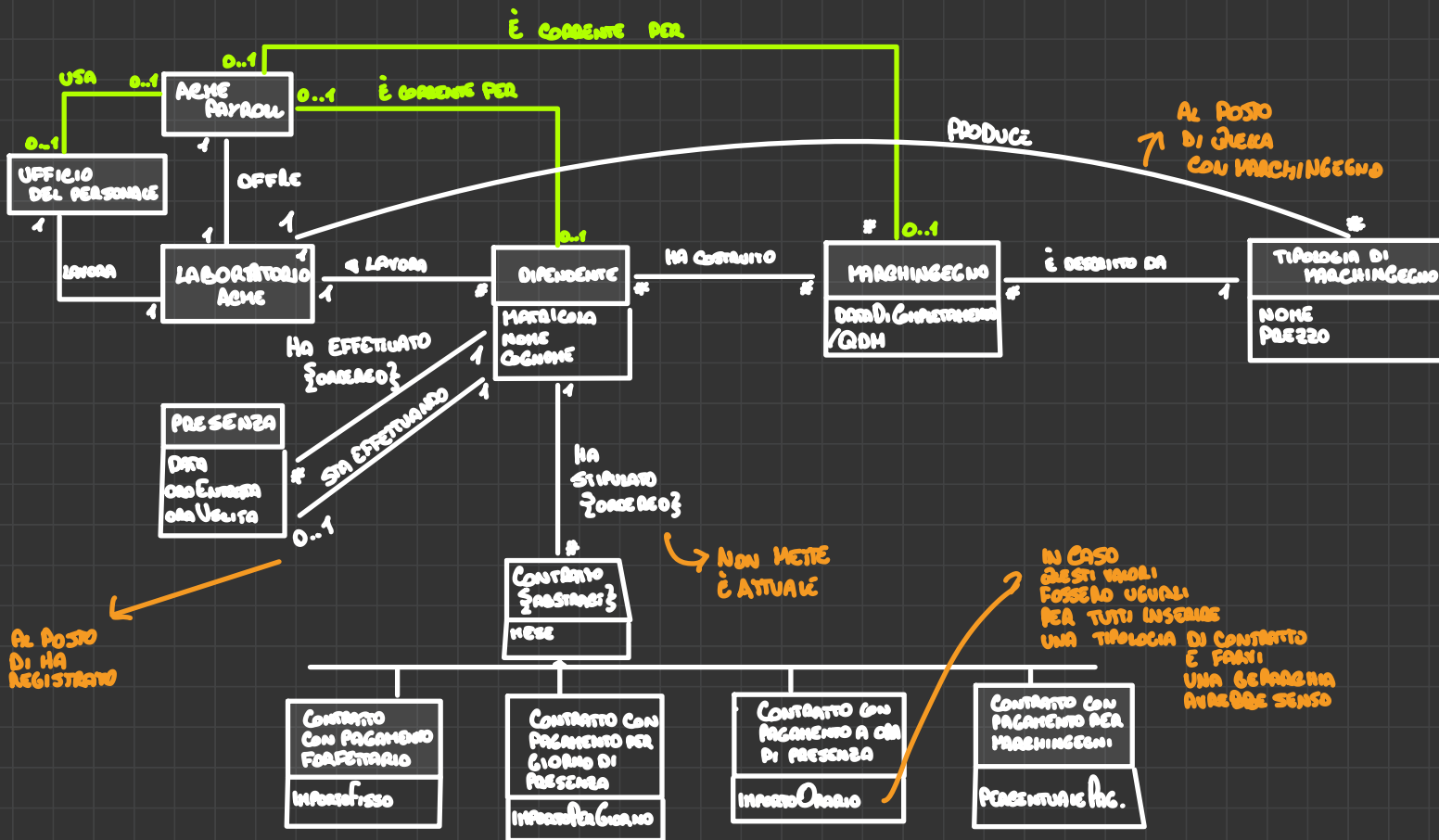
- Mostrare il diagramma delle classi di progetto che si intende realizzare relativamente ai casi d'uso UC1-UC3. In particolare, mostrare le classi e le interfacce che si intendono utilizzare, nonché gli attributi e le associazioni (con le relative molteplicità e navigabilità).

Esercizio A3 (Progettazione)

Considerando per questo esercizio tutti i tipi di contratti per il pagamento dei dipendenti, fare la progettazione orientata agli oggetti per il sistema in discussione, relativamente al caso d'uso UC4 (Pagamento dipendente), come segue:

- Mostrare i diagrammi di interazione relativi al calcolo dell'importo da pagare ad un dipendente per il lavoro svolto nell'ultimo mese.

Modello di Dominio

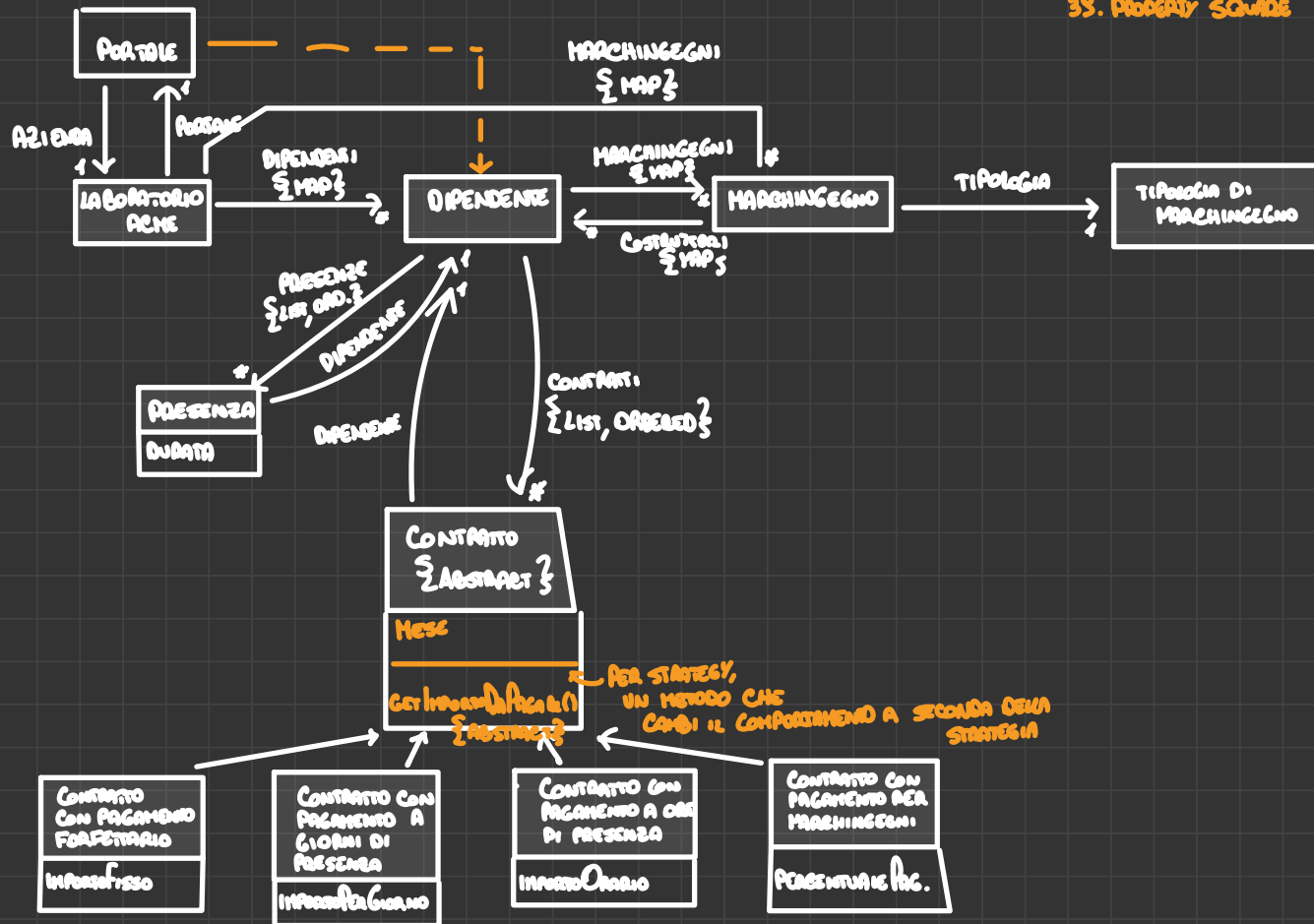


DCD

SQUARE

PRICING STRATEGY

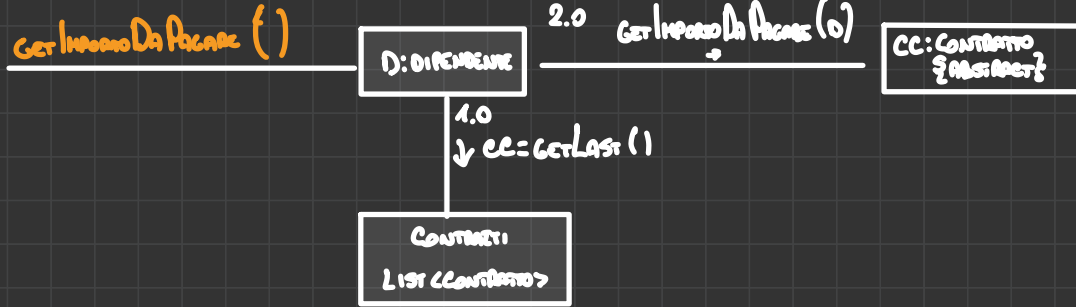
39. PROPERTY SQUARE



Diagrammi di Interazione

Calcolo Stipendio per Dipendente

Il calcolo dello stipendio lo fa il dipendente, non ci interfacciamo con il contabile

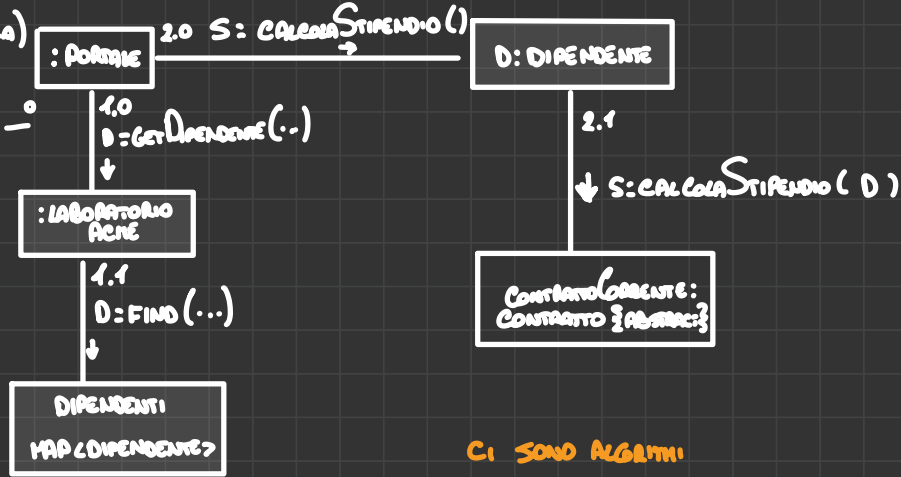


IL CALCOLO DELLO STIPENDIO
 LO FA IL
 DIPENDENTE, NON CI INTERFACCIAVO CON IL CONTROLLO

GET LAVORNO DA PAGARE

CALCOLO STIPENDIO DA DIPENDENTE (INTERFACCIA)

CONTROLLO



CI SONO ALGORITMI

DEGASI PER FARE LA STESSA COSA

STRATEGY

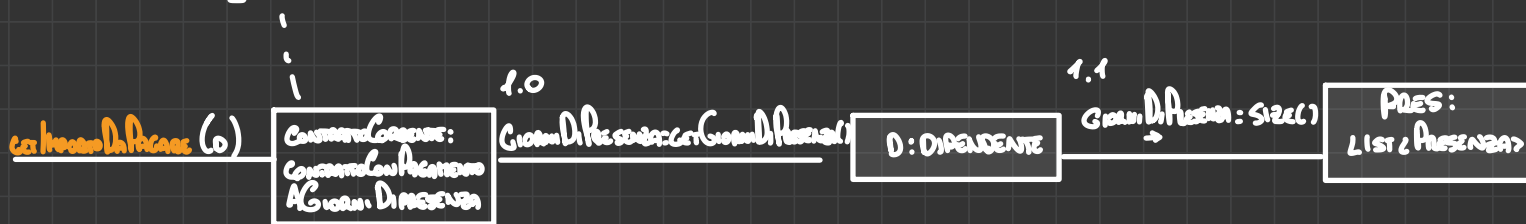
OGGETTO STRATEGIA: I CONTRATTI

OGGETTO CONTESTO: DIPENDENTE

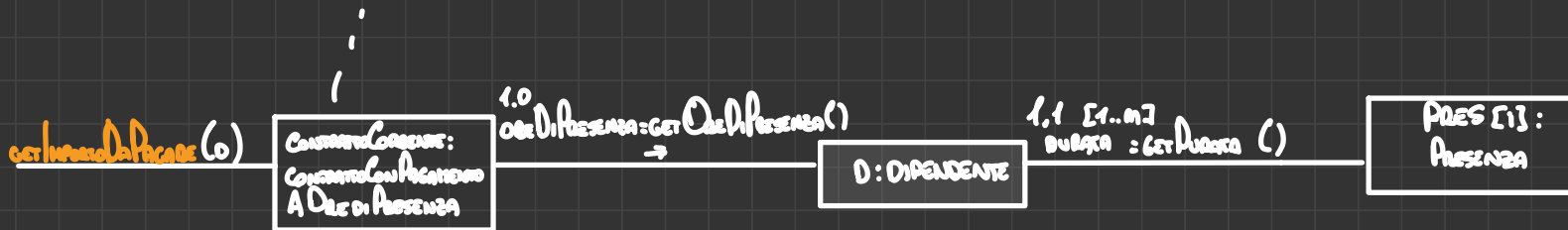
CALCOLO STIPENDIO

GET IMPORNO DA PRESENZA (0) Constante Coefficiente:
Constante Coefficiente
For Fatturato , { RETURN THIS.IMPORNO FOR FATTORIO }

{ SHOW THIS.IMPORNO * GIORN DI PRESENZA }



{ RETURN THIS.IMPORNO ORE * ORE DI PRESENZA }



§ RETIAMI THIS PER LEARNING MACHINE E TOTALE QDM §

calcolaStruttura (o)

CONSTRUTTORE:
CONSTRUTTORE PER
PER MACHINE E CNO

D: DIPENDENTE

1.1 [1..n]

QDM [i] = GET QDM (i)

MACC [i]:
MACHINE E CNO

1.1.1

numDip: size (i)

CONSTRUTTORE:
MAP <DIPENDENTE>

1.1.2

P: GET PREZZO (i)

TIPOLOGIA:
TIPOLOGIA DI MACHINE E CNO

1.0

TOTALE QDM: GET TOTALE QDM (i)

§ RETIAMI P / numDip §

